

Security Handbook : CDN and Front-End Supply Chain

01

Security Handbook Series

GRANT SUPPORT • 2025-2026



Supported by Ethereum Foundation's
Ecosystem Support Program (ESP)



Contents

Frontispiece	4
About This Handbook	4
Copyright and License	5
Author and Creative Credits	5
Purpose and Scope	6
Target Audience	6
How to Use This Handbook	6
Relationship to SCSVS, SCSTG, and SCWE	6
Part I: Foundations	8
1. Introduction to Front-End and Supply Chain Security	8
2. Threat Model Overview	11
Part II: CDN and Content Delivery	14
3. CDN Security	14
4. Content and Asset Integrity	17
Part III: Front-End Supply Chain	24
5. Dependency Awareness	24
6. Build and Pipeline Security	31
7. Third-Party Scripts and Widgets	36
Part IV: Threat Models by Component	41
8. Threat Model: CDN	41
9. Threat Model: Front-End Application Stack	42
10. Common Front-End Vulnerabilities (Web and Mobile)	42
11. Threat Model: NPM/Package Ecosystem	43
12. Threat Model: Build and Deploy Pipeline	44
13. Threat Model: Runtime (Browser, Mobile)	45
14. Mobile Application Security (Web3 Front-Ends)	46
15. Security Tools and Resources for Front-End	46
Part V: Mitigations and Operations	47
16. Checklists and Controls	48
17. Monitoring and Detection	53
18. Incident Response for Supply Chain Compromise	56

Part VI: Appendices 60

19. Appendix A: Glossary 60

20. Appendix B: Supply Chain Threat Model Templates 65

21. Appendix C: References and Further Reading 72

Appendix C: References and Further Reading 73

Standards and Frameworks 73

Incidents and Case Studies 74

Tools and Documentation 74

Further Reading 75

Frontispiece

OWASP SMART CONTRACT SECURITY PROJECT

CDN and Front-End Supply Chain Security

About This Handbook

CDN and Front-End Supply Chain Security is part of the OWASP Smart Contract Security Handbook Series. The series extends smart contract security beyond contract code into the people, process, application, infrastructure, and operational layers surrounding Web3 systems.

Each handbook is designed as a defensive field reference for architects, engineers, security practitioners, incident responders, auditors, and organizational leaders. It complements the OWASP Smart Contract Security Verification Standard (SCSVS), Smart Contract Security Testing Guide (SCSTG), Smart Contract Weakness Enumeration (SCWE), Smart Contract Top 10, and SCS Checklist without replacing those resources.

SERIES EDITION **2026**

Copyright and License

Copyright © 2026 The OWASP Foundation and contributors.



This document is released under the [Creative Commons Attribution-ShareAlike 4.0 International License](#). You may share and adapt the material with appropriate attribution and under the same license. For reuse or distribution, make the license terms clear and identify material changes.

OWASP publications are community-developed educational resources. References to organizations, products, or services do not constitute endorsement, and the material is provided without warranty.

Author and Creative Credits

PROJECT LEAD

Shashank | CredShields

CEO and Co-Founder

AUTHOR | PROJECT MAINTAINER

Pratik Lagaskar

Security Researcher

COVER PAGE DESIGNS

Siddharth Neekher

Lead UI Designer @ CredShields

OWASP Smart Contract Security Project

Purpose and Scope

Threat models and defensive guidance for the content delivery network (CDN), front-end, and supply chain components of Web3 applications. The handbook is grounded in the controls that matter today: Subresource Integrity (SRI), Content Security Policy (CSP), Software Bill of Materials (SBOM), Supply-chain Levels for Software Artifacts (SLSA), and signing with Sigstore, set against real CDN and dependency compromises such as the Polyfill.io attack of June 2024, the Ledger Connect Kit incident of December 2023, and the npm and CI/CD attacks of 2024 and 2025. It covers dependency management, the build and deployment pipeline, content integrity, and third-party script and asset risk, with the regulatory context of U.S. Executive Order 14028 and the EU Cyber Resilience Act.

A dApp's smart contracts can be flawless while its users still lose funds, because the code a user actually runs is the JavaScript a CDN serves to their browser. That front end is the largest and least-audited attack surface in Web3. This handbook treats it as a first-class security boundary.

Target Audience

Front-end and full-stack developers, DevOps and platform engineers, and security practitioners responsible for dApp front ends and their delivery infrastructure.

How to Use This Handbook

Read Part I to fix the threat model and vocabulary. Parts II through IV are reference material organized by component: the CDN and content layer, the front-end supply chain, and per-component threat models. Part V is operational: checklists, monitoring, and incident response for a supply chain compromise. The appendices hold a glossary, reusable threat-model templates, and the consolidated bibliography. Engineers shipping a change tonight can jump to the checklists in Part V and the SRI and CSP sections in Part II.

Relationship to SCSVS, SCSTG, and SCWE

This handbook complements the OWASP smart contract standards rather than duplicating them. The Smart Contract Security Verification Standard (SCSVS) defines what a secure system must satisfy; the front-end controls here are the delivery-layer counterpart to its integrity requirements. The Smart Contract Security Testing Guide (SCSTG) describes how to test; the pipeline and dependency tests in Parts III and V extend that testing outward to the build system. The Smart Contract Weakness Enumeration (SCWE) catalogs contract weaknesses; the front-end and supply-

chain weaknesses here fill the gap the enumeration leaves at the browser and build boundary. The companion Web3 Attack Vectors Mapping Handbook (10) places these risks in the broader WA01 to WA15 taxonomy, and the Incident Response Handbook (06) owns the response process this handbook feeds into.

Part I: Foundations

This part fixes the vocabulary and the threat model used throughout the handbook. It explains why the front end is a security boundary equal in weight to the smart contracts behind it, breaks the supply chain into four levels so that later controls have a place to attach, and maps the attack surface with STRIDE so nothing is analyzed by intuition alone.

1. Introduction to Front-End and Supply Chain Security

1.1 Why Front-End and CDN Matter for Web3

The security conversation in Web3 concentrates on the smart contract, because that is where the money is held. The contract, though, is not what a user interacts with. A user opens a web page, that page loads JavaScript, and that JavaScript constructs the transaction, populates the recipient and amount, and asks the wallet to sign. If any byte of that JavaScript is attacker-controlled, the signed transaction is attacker-controlled, and no amount of on-chain audit rigor prevents the loss. The front end is the first and last thing a user touches, and it is delivered over infrastructure the protocol team rarely owns end to end.

Two properties make this surface uniquely dangerous in Web3. First, transactions are irreversible: a drained wallet has no chargeback, so a front-end compromise converts directly and permanently into stolen funds. Second, the delivery path is long and shared. Front-end code passes through package registries, build systems, artifact stores, CDNs, and the browser before it runs, and each hop is a place where an adversary can substitute code. The [Ledger Connect Kit compromise of December 2023](#) is the canonical example: an attacker phished a former employee's npm credentials, published a malicious version of a widely embedded connector library, and drained wallets across many dApps that had done nothing wrong in their own code. The contracts were fine. The delivery chain was not.

The CDN deserves special attention because it is trusted by default and rarely re-verified. A `<script src="https://cdn.example/lib.js">` tag says “run whatever this host sends, forever.” When [Funnull](#) acquired the [polyfill.io](#) domain in February 2024 and began serving malicious

redirects by June, hundreds of thousands of sites that had embedded `cdn.polyfill.io` years earlier started shipping attacker code to their users without changing a single line of their own source. Trust that is granted once and never checked again is the recurring failure mode of this layer.

1.2 Supply Chain Levels (Code, Build, Deploy, Runtime)

To reason about defenses, split the software supply chain into four levels. Each level is a distinct trust boundary with its own adversaries, its own evidence of integrity, and its own controls. A compromise at any level flows downstream to the user unless a later level independently verifies what it received.

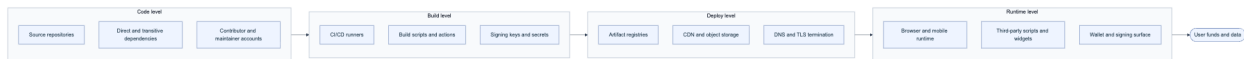


Figure 1. The four supply chain levels. Integrity evidence created at one level, such as a signed provenance attestation at the build level, is only useful if a downstream level verifies it.

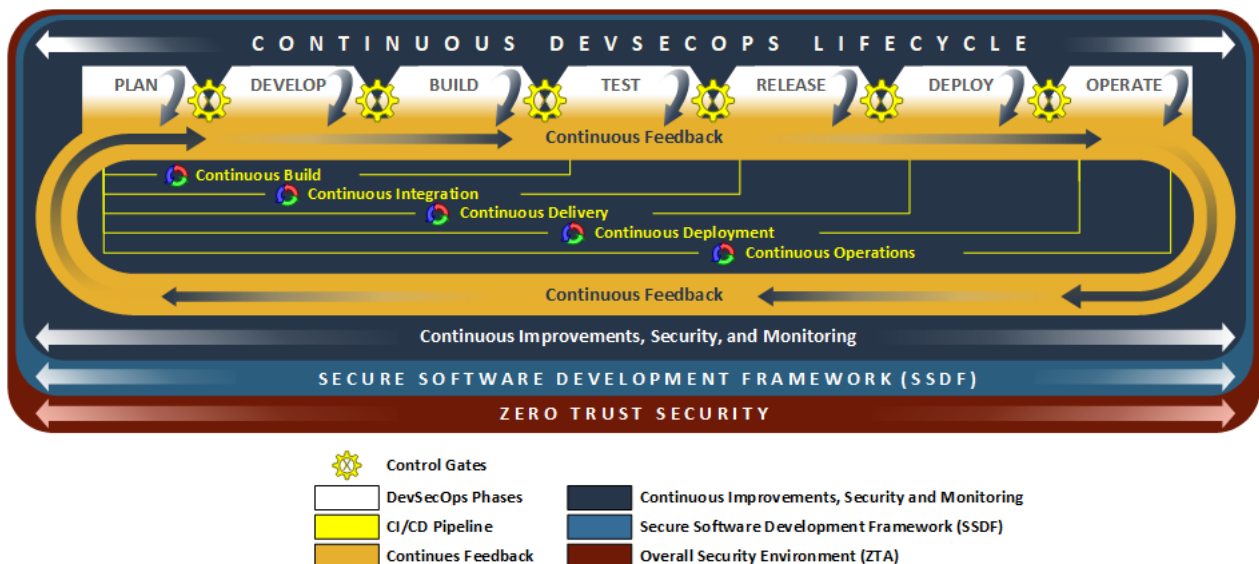


Figure. NIST's notional DevSecOps reference model puts the four-level abstraction above into an operational lifecycle. The important difference is that security and continuous monitoring cross every phase; they are not a final scan performed after an artifact is already ready to deploy. Source: [NIST NCCoE, DevSecOps Notional Reference Model](#), public domain (U.S. government work).

The **code level** covers everything that exists before a build runs: your first-party source, the direct and transitive dependencies you pull from npm or another registry, and the human and machine accounts that can write to any of them. Attacks here include typosquatting, dependency confusion, malicious maintainer takeover, and the slow social-engineering path taken by the [XZ Utils backdoor](#), where an adversary contributed to an open-source project for over two years to earn the maintainer trust needed to insert a backdoor into official releases.

The **build level** is where source becomes artifact. The runner, the build scripts, the third-party build actions, and the secrets available during the build are all in scope. The [tj-actions/changed-files compromise of March 2025 \(CVE-2025-30066\)](#) targeted this level: a popular GitHub Action used in more than 23,000 repositories was altered so that its version tags pointed to a commit that dumped the runner's memory, spilling CI/CD secrets into workflow logs.

The **deploy level** is where the artifact is published and served: artifact registries, the CDN, object storage, and the DNS and TLS that route users to it. The Polyfill.io attack lived here. The **runtime level** is the browser or mobile app where the code finally executes, alongside every third-party script the page pulls in and the wallet-signing surface the user ultimately confirms on.

1.3 Relationship to SCSVS, SCSTG, and SCWE

The OWASP smart contract standards define the contract side of a Web3 system. This handbook is the delivery-side complement, and the mapping is deliberate. The [Smart Contract Security Verification Standard \(SCSVS\)](#) states verifiable requirements a system must meet; the SRI, CSP, SBOM, and provenance controls in this handbook are the front-end requirements that sit alongside it, so that “the system is secure” is not quietly narrowed to “the bytecode is secure.” The [Smart Contract Security Testing Guide \(SCSTG\)](#) describes test procedures; Parts III and V here extend testing to the dependency graph and the build pipeline, which the contract-focused guide does not reach. The [Smart Contract Weakness Enumeration \(SCWE\)](#) catalogs contract weaknesses; front-end and supply-chain weaknesses are the class it does not enumerate, and this handbook fills that space. The [OWASP Web3 Attack Vectors Top 15](#) frames supply-chain attacks (WA02) and UI/UX deception (WA06) as first-tier non-contract risks, and the companion Handbook 10 maps them in full.

1.4 How to Use This Handbook

Read Part I once to internalize the four-level model and the STRIDE mapping. After that, treat the handbook as reference. When you are hardening the CDN and content layer, work from Part II; when you are securing dependencies and the build, work from Part III; when you are threat-modeling a specific component, jump to the matching chapter in Part IV. Part V is the operational core: its checklists are meant to be lifted directly into a release runbook, and its incident-response section is meant to be read before an incident, not during one. Cross-references point to the standards and to sibling handbooks so you can follow a thread outward when a topic touches DNS (02), infrastructure (07), SDKs (08), or the response process (06).

2. Threat Model Overview

2.1 Attack Surfaces: CDN, Front-End, Dependencies, Build

A threat model starts by enumerating what an adversary can reach. For a Web3 front end, the reachable surfaces cluster into four groups that mirror the supply chain levels but are framed as attacker entry points rather than trust boundaries.

The **CDN and content surface** is every host that serves bytes to the browser: the primary CDN, any secondary or fallback CDN, object storage buckets behind them, and the DNS names and TLS certificates that make those hosts trusted. An attacker who controls any of these can substitute or tamper with delivered content. Entry points include a compromised CDN account, a lapsed or hijacked domain, cache poisoning, and a mis-issued certificate.

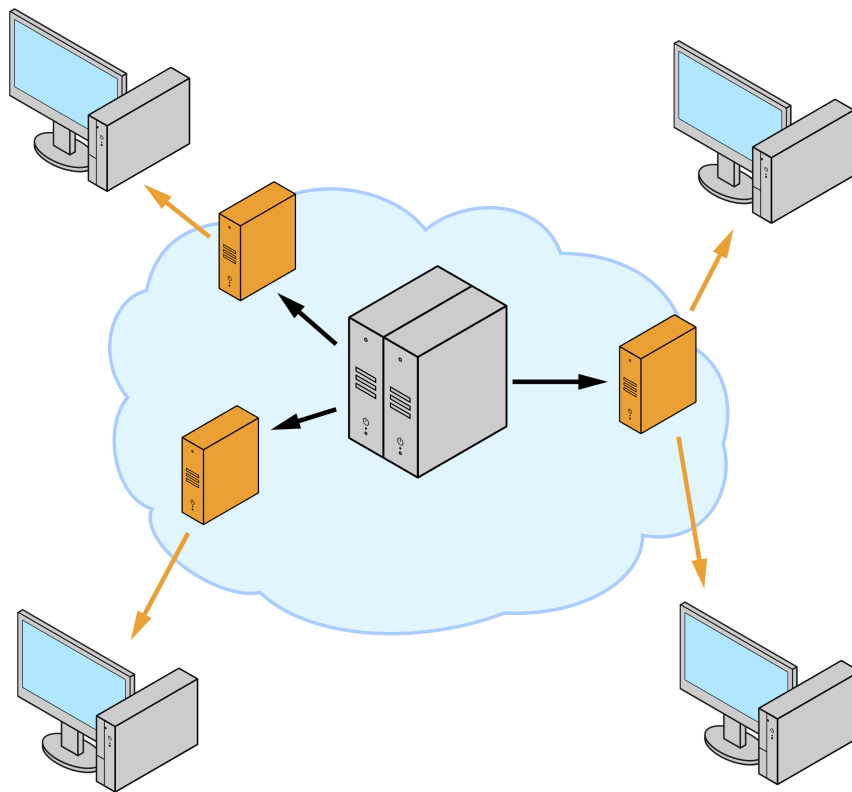


Figure. A generalized content distribution network, where proxy and cache servers between the origin and the end user each become a host capable of serving, and therefore substituting, delivered content. Source: [Wikimedia Commons](#), CC BY-SA 4.0.

The **front-end application surface** is the first-party code and the way it composes third-party code at runtime: inline scripts, dynamically injected scripts, `iframe` embeds, and browser storage. Cross-site scripting, DOM clobbering, and prototype pollution live here, and in a wallet context they lead directly to transaction manipulation rather than mere session theft.

The **dependency surface** is the transitive graph of packages the application pulls in. npm projects routinely resolve to thousands of transitive packages, any one of which can be typosquatted, confused with an internal name, or taken over by a compromised maintainer. The **build surface** is the CI/CD system, the third-party actions it invokes, the secrets it holds, and the artifact stores it writes to. These last two are the surfaces an attacker prefers, because a single successful compromise there is amplified across every user of every downstream project.

2.2 Threat Actors and Objectives

Not every adversary wants the same thing, and the control set should reflect who you are defending against. Four actor profiles cover most real incidents in this space.

Actor	Capability and access	Primary objective	Representative incident
Financially motivated criminal	Buys or phishes registry, CDN, or domain credentials; runs typosquat and dependency-confusion campaigns at scale	Drain wallets, harvest keys and seed phrases	Ledger Connect Kit (Dec 2023); Polyfill.io (Jun 2024)
Nation-state actor	Patient social engineering, long-horizon maintainer takeover, custom malware, targeted CI/CD intrusion	Espionage, sabotage, revenue generation under sanctions	XZ Utils backdoor (2024); DPRK campaigns against exchange staff
Malicious or negligent insider	Legitimate access to source, build, or deploy that outlives their role	Theft, sabotage, or accidental exposure	Former-employee credential reuse in Connect Kit
Opportunistic automated scanner	Mass scanning for exposed secrets, unpinned actions, mutable tags	Widen access, sell or reuse harvested credentials	Automated exploitation following tj-actions disclosure

The objective column matters for prioritization. A financially motivated actor needs only one weak hop to monetize, so the marginal value of hardening the weakest link is high. A nation-state actor invests in the build and maintainer levels, which is where provenance and signing pay off. An insider defeats perimeter controls, so revocation timeliness and least privilege (covered in the Employee Lifecycle Handbook, 03) are the relevant defenses.

2.3 STRIDE and Scenario Mapping

STRIDE gives the model discipline: for each surface, ask which of the six threat categories applies and what the concrete scenario is. Working the matrix prevents the common failure of hardening the surface you already worry about while ignoring an adjacent category.

STRIDE category	Front-end and supply-chain scenario	Primary control
S poofing	Attacker serves malicious JS from a look-alike or hijacked CDN host or domain	SRI, domain and registrar lock, DNSSEC (Handbook 02)
T ampering	Delivered script, dependency, or build artifact is modified in transit or at rest	SRI, SBOM, signed provenance (SLSA), reproducible builds
R epudiation	No record of who published which artifact, so a malicious release cannot be attributed	Signed commits and releases, Sigstore transparency log (Rekor)
I nformation disclosure	CI secrets, API keys, or private RSA keys leak from a build runner	Secret scanning, least-privilege tokens, pinned actions
D enial of service	CDN, RPC, or dependency registry is made unavailable, breaking the dApp	Multi-CDN, provider redundancy, offline fallbacks (Handbook 07)
E levation of privilege	Third-party script or dependency gains the same execution rights as first-party code	CSP with <code>strict-dynamic</code> , sandboxing, dependency review

A worked scenario ties the categories together. Consider a dApp that embeds an analytics script from a third-party CDN. Spoofing risk: the analytics domain expires and is re-registered by an attacker. Tampering risk: the analytics vendor is breached and ships modified JS. Elevation risk: because the script runs in the page's origin, it can read the DOM of the swap form and rewrite the recipient address before signing. The controls that cut every branch are SRI on the script tag (pins the exact bytes), a CSP that denies the third party the ability to load further scripts, and a design that never renders untrusted content into the signing path. Later parts turn each of these into concrete configuration.

The reusable STRIDE and threat-model templates in [Appendix B](#) let you run this same analysis against your own architecture.

Part II: CDN and Content Delivery

The content delivery layer is where a Web3 front end meets infrastructure the team usually does not own. This part covers how to choose and trust a CDN, how to guarantee that the bytes a user receives are the bytes you published, and how the two canonical failures of this layer, CDN compromise and cache poisoning, actually work. The centerpiece is Subresource Integrity, the one control that turns “trust the CDN forever” into “trust the CDN only as long as it serves the exact hash I pinned.”

3. CDN Security

A content delivery network caches copies of your static assets at edge locations near users and answers their requests from the nearest edge. That is a performance and availability win, and it is also a delegation of trust: for the duration of the cache, the CDN, not your origin, decides what bytes a user runs.

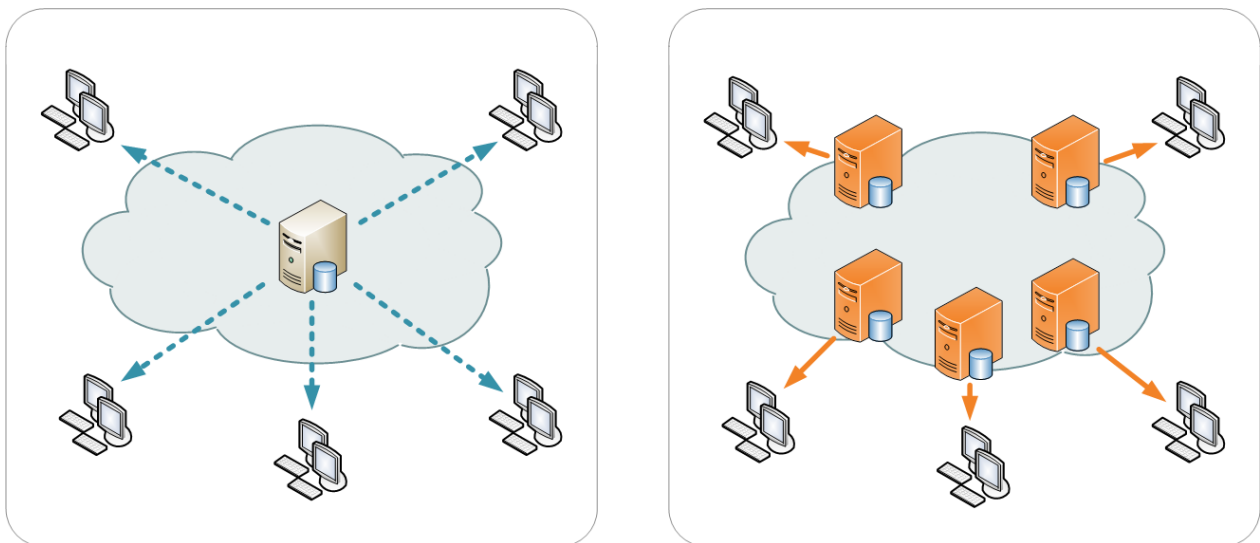


Figure 2. A CDN routes each user to a nearby edge node that serves a cached copy of the origin's content. Every edge, and the control plane behind it, is part of the front end's trust boundary. Source: [Wikimedia Commons](#), CC BY-SA 3.0.

3.1 CDN Provider Selection and Trust

Selecting a CDN is a trust decision, not only a latency benchmark. The provider can read and rewrite everything it serves, so its account security, its insider controls, and its history of handling compromises are part of your security posture. Evaluate providers on concrete, checkable criteria: whether the account supports hardware-backed multi-factor authentication and enforces it for every operator, whether API tokens can be scoped and short-lived, whether configuration changes are logged immutably and exportable to your own monitoring, and whether the provider publishes post-incident reports when it fails. A provider that cannot answer how a stolen operator token would be detected is telling you the answer is “it would not be.”

Trust is also a function of blast radius. A CDN that also terminates your TLS holds your private key material or a delegated certificate, which raises the stakes of an account compromise. A CDN that only serves already-signed, integrity-pinned assets holds far less power over your users. Prefer architectures where the CDN’s authority is bounded by controls you enforce independently, so that a provider breach degrades gracefully rather than becoming a full front-end takeover.

3.2 Content Integrity (SRI, Signing, Pinning)

Content integrity means a user can prove that delivered content matches what you published, without trusting the delivery path. Three mechanisms provide it at different layers. Subresource Integrity (covered in depth in Chapter 4) lets the browser reject a script or stylesheet whose hash does not match a value you embedded in your HTML. Cryptographic signing (Sigstore and Co-sign, Section 4.3) lets a verifier confirm that an artifact was produced by a known identity and recorded in a public log. Pinning, in this context, means committing to exact versions and exact hashes rather than mutable references such as `latest` or a floating major version.

The controls compose. SRI protects the last hop to the browser but says nothing about who built the file; signing proves provenance but is not enforced by the browser at load time; pinning ensures the thing you signed and the thing you reference are the same thing. A front end that uses all three closes the gap that any one leaves open.

3.3 Cache Poisoning and Manipulation

A CDN cache is a shared store keyed by some subset of each request, called the cache key. If an attacker can influence a part of the response that is not part of the key, that is an *unkeyed input*, they can store a malicious response under a key that innocent users will hit. This is web cache poisoning, mapped in detail by [James Kettle’s PortSwigger research](#). Headers such as `X-Forwarded-Host`, `X-Original-URL`, and `X-Rewrite-URL` frequently influence a response while being excluded from the cache key, so a single crafted request can poison the cached page for everyone who follows.

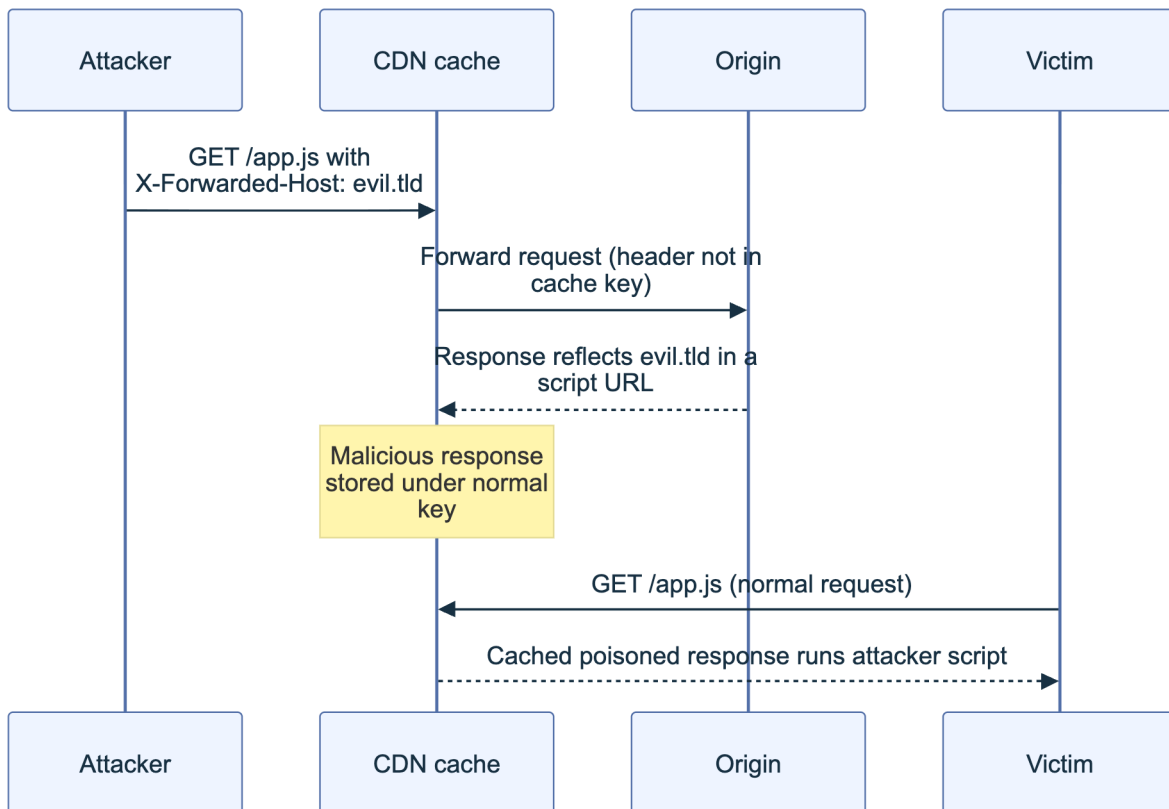


Figure 3. Web cache poisoning through an unkeyed header. The attacker pollutes the shared cache once; every subsequent victim receives the tampered asset.

The defenses are structural. Normalize or strip request headers that the origin does not intend to honor, so they cannot reflect into responses. Include any header that legitimately varies the response in the cache key (the `Vary` mechanism, though note that some CDNs handle it inconsistently). Avoid reflecting request-controlled values into cacheable responses at all. Discovery tooling such as Kettle’s Param Miner finds unkeyed inputs before an attacker does. Crucially, SRI is a backstop here as well: a poisoned script that does not match its pinned hash is rejected by the browser regardless of how it entered the cache.

3.4 TLS and Origin Protection

TLS protects content in transit between the edge and the user, and separately between the CDN and your origin. Both legs matter. Terminate TLS with modern protocol versions (TLS 1.3 preferred, TLS 1.2 minimum), disable legacy ciphers, and enable HTTP Strict Transport Security (HSTS) so browsers refuse to downgrade to plaintext. Protect the origin from direct access that bypasses the CDN: if an attacker can reach your origin IP directly, CDN-layer protections and WAF rules are moot. Enforce origin access with authenticated pull (the CDN presents a secret or client certificate the origin requires), IP allowlisting of CDN egress ranges, or mutual TLS. Bind

certificates with Certificate Authority Authorization (CAA) records so that only your chosen CAs can issue for your domain, a control that ties into the DNS hardening covered in the DNS and Hosting Handbook (02).

3.5 Geographic and Availability Considerations

A CDN is also an availability control, and availability is a security property when a dApp's front end is the only path to user funds. Distribute across regions so that a regional outage does not take the application offline, and understand the failure behavior when the CDN itself is unavailable. Two design choices recur throughout this handbook: consolidation on a single provider lowers the attack surface but concentrates outage and compromise risk, while diversification across multiple providers adds resilience at the cost of a larger surface and harder integrity guarantees (multi-CDN means multiple accounts, multiple configs, and multiple places to pin hashes). The Infrastructure Security Handbook (07) treats this single-versus-multi-provider tradeoff as a first-class decision. For most teams, a primary CDN with an independently controlled static fallback (for example, an IPFS-pinned copy referenced by an integrity hash) balances the two.

4. Content and Asset Integrity

Integrity is the property that the code running in a user's browser is exactly the code you intended to ship. This chapter makes that property concrete, then works a real failure where it was absent.

4.1 Subresource Integrity (SRI) (OWASP, MDN)

Subresource Integrity is a browser feature that lets you pin the cryptographic hash of an external script or stylesheet in your HTML. Before executing the resource, the browser hashes the fetched bytes and compares them to your pinned value; on mismatch, it refuses to run the resource. This converts an unconditional "trust this CDN" into a conditional "trust this CDN only while it serves these exact bytes." The [MDN Subresource Integrity reference](#) and the [OWASP guidance](#) define the mechanism.

4.1.1 SRI Hashes (SHA-256/384/512) and integrity Attribute

You express the pin with the `integrity` attribute, whose value is a hash algorithm prefix followed by the base64-encoded digest of the resource. SRI supports SHA-256, SHA-384, and SHA-512. Pair it with `crossorigin="anonymous"` so the CORS-enabled fetch provides the response bytes the browser needs to validate.

```
<script
  src="https://cdn.example.com/wallet-connect.v3.2.1.js"
  integrity="sha384-oqVuAfXRKap7fdgcCY5uykM6+R9GqQ8K/uxy9rx7HNQlGYl1kPzQho1wx4JwY8wC"
  crossorigin="anonymous"></script>
```

Generate the hash from the exact file you intend to ship:

```
# Produce the integrity value for a released asset
cat wallet-connect.v3.2.1.js | openssl dgst -sha384 -binary | openssl base64 -A
# Prefix the output with "sha384-" to form the integrity attribute value
```

You may supply multiple hashes separated by spaces; the browser accepts the resource if it matches any, and prefers the strongest algorithm present. Use this to roll hashes during a planned content change without a flash of broken loads.

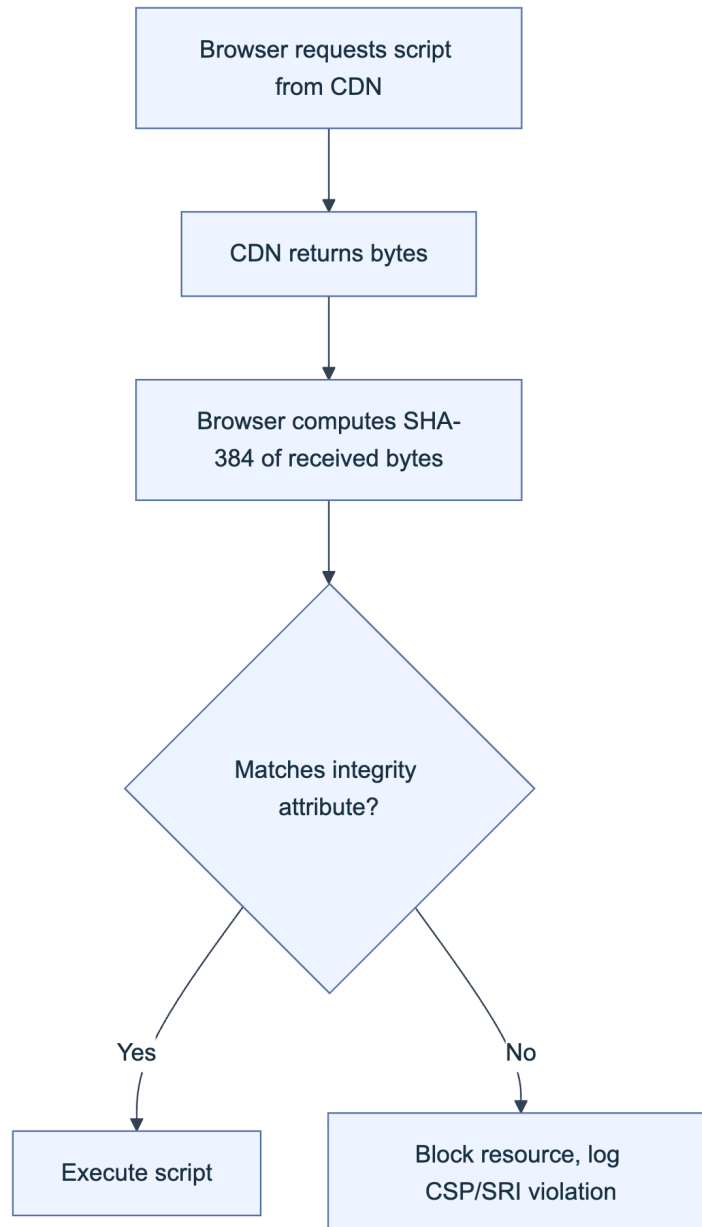


Figure 4. SRI verification at load time. A tampered or substituted file fails the hash check and never executes.

4.1.2 Protection Against CDN Compromise and In-Transit Tampering

SRI directly defeats two important failure modes at this layer. If the CDN is compromised and serves a modified `wallet-connect.js`, the modified bytes do not match the pinned hash and the browser blocks them. If a network position between the edge and the user tampers with the response, the same check fails. This is exactly the class of attack that hit Polyfill.io and Ledger Connect Kit, and in both cases an SRI pin on the embedding pages would have converted a silent wallet-drain into a visible, blocked resource. SRI is therefore one of the highest-leverage controls in this handbook, but it does **not** eliminate trust in the first-party document or deployment control plane: an attacker who can rewrite the embedding HTML can replace the script URL and its integrity metadata together. Protect the HTML, deployment credentials, and release authorization with the provenance and dual-control measures in Parts 3 and 5.

4.1.3 Limitations: Version Pinning and Hash Updates; Not for Inline Scripts or Dynamic Content

SRI is not free of sharp edges. It pins a specific version: any legitimate update to the file changes its hash, so a CDN that silently updates a floating version will break every page that pinned the old hash. This is a feature (it forces you to acknowledge changes) but it means SRI only works with immutable, versioned URLs, never with `latest`. SRI does not cover inline scripts or content generated at runtime, and it does not apply to resources loaded by an already-running script unless that script implements its own checks. It also cannot validate content that legitimately varies per request. The practical rule: serve third-party code from immutable, versioned URLs, pin each with SRI, and use CSP (Section 4.5) to constrain everything SRI cannot reach.

4.2 Case Study: Polyfill.io and CDN Supply Chain Compromise

Polyfill.io is the reference incident for this entire handbook. Polyfill.io was a widely embedded service that delivered browser “polyfill” scripts, and site operators referenced it with a plain `<script src="https://cdn.polyfill.io/...">` tag, no integrity pin. In February 2024, a company named Funnnull acquired the domain and its associated GitHub account. For months the service behaved normally, then in June 2024 it began serving malicious code that redirected mobile users to scam and malware destinations, injected conditionally to evade detection ([Sansec disclosure, 25 June 2024](#)).

The scale was large and the timeline instructive:

Date	Event
Feb 2024	Funnnull acquires the polyfill.io domain and GitHub account
Jun 25, 2024	Sansec discloses active malicious injection
Jun 25-27, 2024	Cloudflare and Fastly stand up safe mirrors ; Google warns advertisers
Jun 27, 2024	Namecheap suspends the malicious domain
Jul 2, 2024	Censys counts 384,773 hosts still embedding the script

Reported impact ranged from the widely quoted 100,000-plus sites up to figures above 490,000, with Cloudflare implementing real-time rewrites of `cdn.polyfill.io` to a safe version to protect users of sites that had not yet updated. The lessons are precisely the controls in this part: an SRI pin would have blocked the malicious bytes; not embedding a mutable third-party CDN reference at all would have removed the dependency; and continuous inventory of third-party script origins (Section 7.3 and Part V monitoring) would have surfaced the stale, now-hostile dependency long before it fired.

4.3 Signed Releases and Verification (Sigstore, Cosign)

SRI proves the browser got the bytes you pinned; signing proves who produced those bytes and when. [Sigstore](#) provides signing that does not require a team to manage long-lived private keys. With keyless signing, the signer authenticates through OpenID Connect, the Fulcio certificate authority issues a short-lived certificate (on the order of ten minutes) binding an ephemeral key to that identity, Cosign signs the artifact, and the Rekor transparency log records an immutable, publicly auditable entry of the event ([OpenSSF overview](#)).

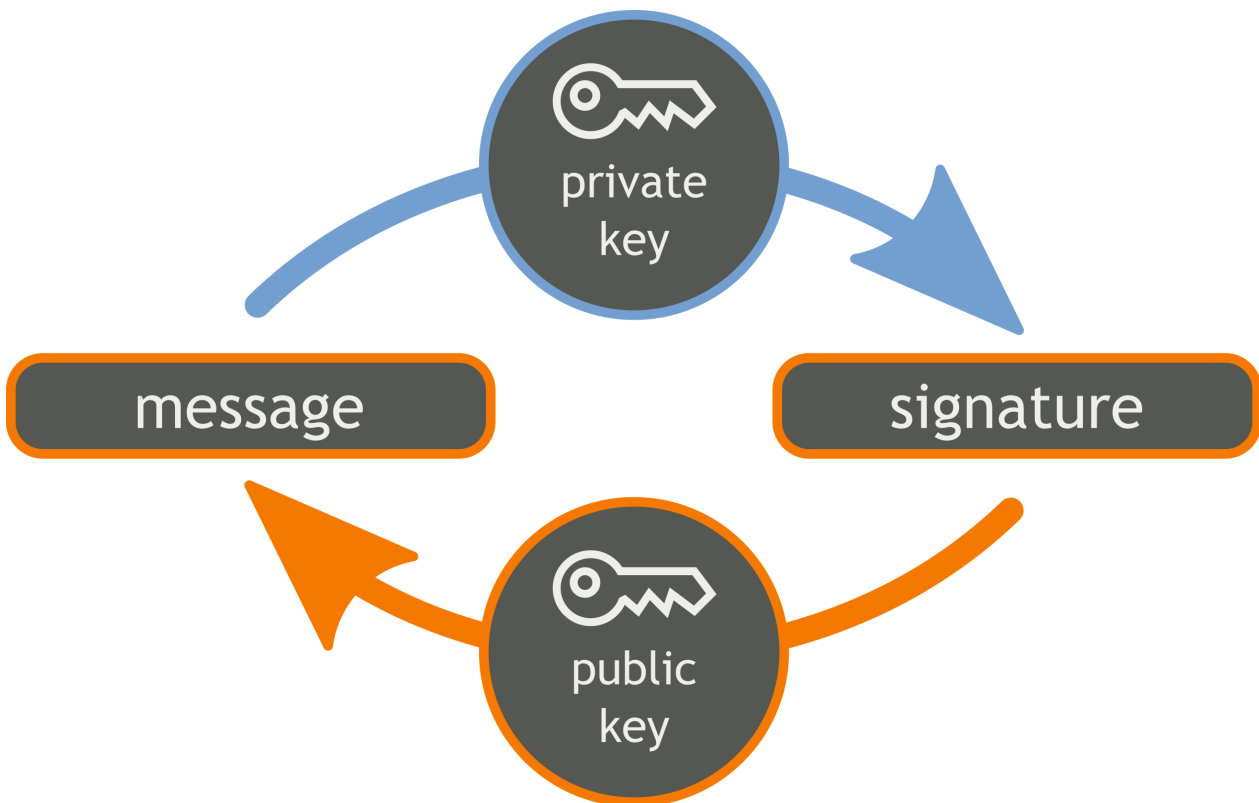


Figure. The underlying primitive Sigstore automates: a private key signs the artifact, and anyone holding the corresponding public key (here, delivered through a short-lived Fulcio certificate rather than a long-lived one) can verify the signature without ever holding the signing secret. Source: [Wikimedia Commons](#), CCO.

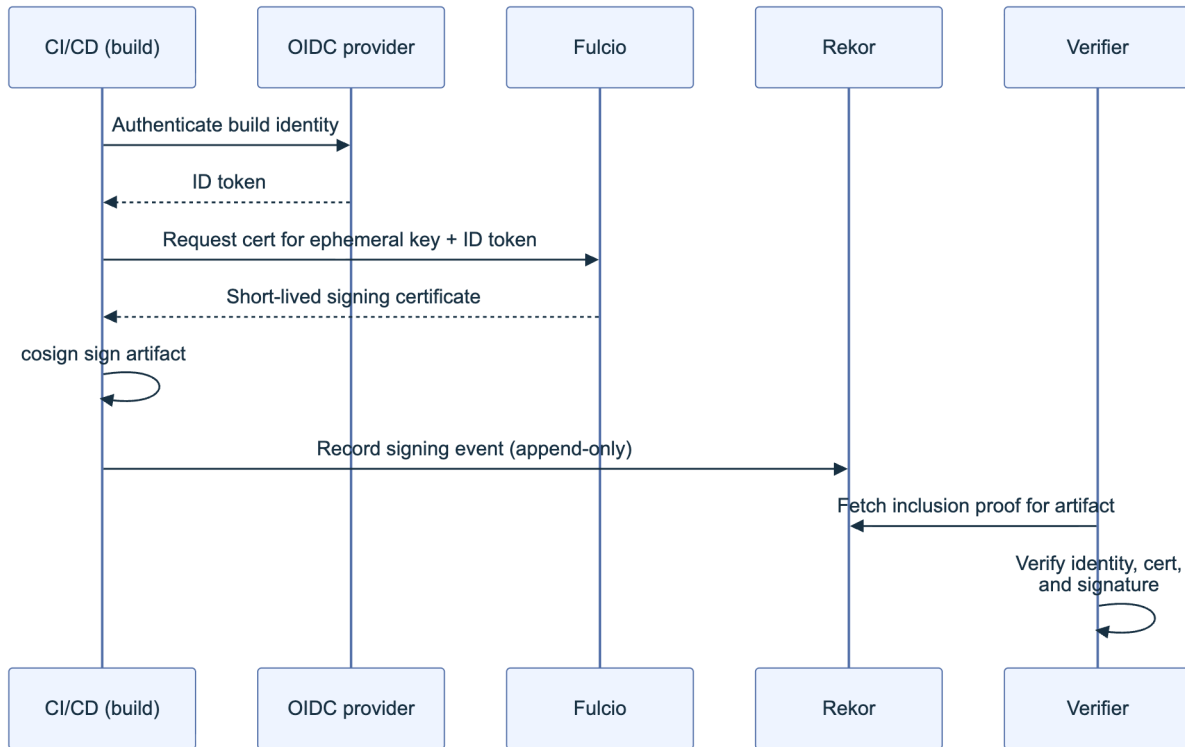


Figure 5. Sigstore keyless signing and verification. The ephemeral key is discarded after signing; trust rests on the identity in the certificate and the public Rekor entry.

For a front end, sign build artifacts (the bundled JS, the SBOM, container images) in CI, publish the signatures, and verify them at the deploy step before anything reaches the CDN. Because Rekor is public and append-only, an artifact owner can monitor the log for their own identity and detect any signing event they did not initiate, which turns silent key misuse into a detectable event.

4.4 Immutability and Versioning

Every integrity control in this part assumes immutable, versioned artifacts. If a URL's content can change under a fixed name, SRI hashes drift, signatures detach from what they signed, and rollback is ambiguous. Adopt content-addressed or strictly versioned URLs (`app.v3.2.1.js`, or a hash-named `app.<digest>.js`), configure long-lived immutable cache headers for them (`Cache-Control: public, max-age=31536000, immutable`), and never reuse a version identifier for changed content. Publish a new version and update the reference and its SRI hash together in the same reviewed change. Immutability also simplifies incident response: rolling back is redeploying an earlier, still-valid artifact, not racing a mutable pointer.

4.5 Content Security Policy (CSP) as Complement

CSP is the enforcement layer for everything SRI and signing do not cover. A Content Security Policy tells the browser which sources may load scripts, styles, images, and connections, and can require integrity on scripts. Modern practice, per [CSP Level 3](#) and [web.dev's strict CSP guidance](#),

is a nonce-based or hash-based policy with `strict-dynamic`, rather than a brittle allowlist of host-names. A per-response nonce marks the scripts you trust; `strict-dynamic` propagates that trust to scripts they load, so you avoid maintaining a fragile domain allowlist while still blocking attacker-injected scripts that lack the nonce.

```
Content-Security-Policy:
  script-src 'nonce-r4nd0m' 'strict-dynamic';
  object-src 'none';
  base-uri 'none';
  require-trusted-types-for 'script';
```

CSP and SRI are complementary: SRI pins the content of the scripts you explicitly reference, while CSP governs what may load at all and contains the damage of any injection, including inline-script XSS that SRI cannot see. The `require-trusted-types-for 'script'` directive further hardens the DOM against injection sinks, which matters acutely on a page whose scripts build financial transactions. Report violations to a monitored endpoint (Part V) so that a blocked resource becomes an alert, not a silent user-facing breakage.

Key controls for Part II

- Pin every third-party script and style with SRI (SHA-384 or SHA-512) served from immutable, versioned URLs.
- Deploy a nonce-based CSP with `strict-dynamic`, `object-src 'none'`, and Trusted Types; report violations.
- Protect the origin from direct access; enforce TLS 1.3 and CAA.
- Sign build artifacts with Sigstore and verify signatures before publishing to the CDN.
- Strip or normalize unkeyed request headers to prevent cache poisoning.

Part III: Front-End Supply Chain

This part moves from the delivery edge to the supply chain that produces what gets delivered: the dependency graph a front end pulls in, the build pipeline that turns source into artifact, and the third-party scripts a page still has to trust at runtime. A modern Web3 front end resolves a dependency tree running into the thousands of transitive packages, and the September 2025 compromise of `chalk`, `debug`, and sixteen other packages downloaded a combined 2.6 billion times a week showed exactly how far one hijacked maintainer account reaches ([Aikido Security](#)). The chapters that follow build a bill of materials for that graph, harden the pipeline that turns it into a shipped artifact, and set rules for the third-party code a page cannot avoid loading.

5. Dependency Awareness

A front end is mostly other people's code. This chapter treats the dependency graph as a first-class attack surface: something to inventory, score, patch on a schedule, and prove the provenance of, rather than something installed once and forgotten.

5.1 Direct and Transitive Dependencies

A direct dependency is a package your `package.json` names explicitly, such as a wallet-connection library or a UI framework. A transitive dependency is everything that package requires to run, and the ones those packages require in turn. A typical Web3 dApp with a handful of direct dependencies routinely resolves to several thousand transitive packages once a wallet SDK, a UI framework, and a build toolchain are all accounted for, because each pulls in its own tree. `npm ls --all` or `npm list --all` prints the full resolved tree; most teams have never run it end to end.

```
$ npm ls --all | wc -l
4128
$ npm ls --all 2>/dev/null | grep -c 'deduped'
612
```

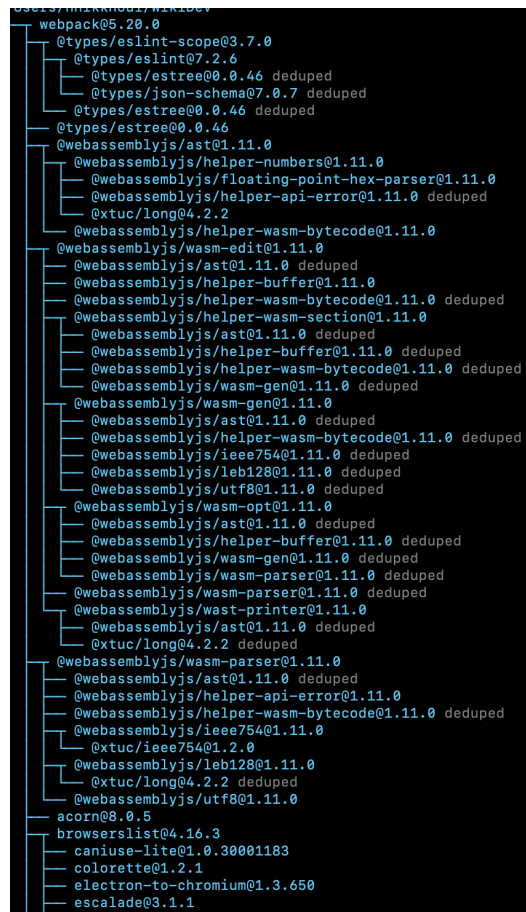


Figure. A real `npm ls`-style dependency tree for a single package, `webpack`, expanded to its transitive dependencies. Each line is a package whose maintainer you have implicitly trusted the moment it resolved into the tree. Source: [Wikimedia Commons](#), CC BY-SA 4.0.

The security consequence is that your trust decision is not “do I trust this library’s authors,” it is “do I trust every maintainer of every package in this tree, transitively, forever.” You audited none of them. The [Ledger Connect Kit incident](#) covered in Part I is a direct dependency compromise; the [XZ Utils backdoor](#) is a transitive one that took over two years of patient social engineering to land. Depth in the tree does not correlate with safety: a package four levels down that nobody on the team has heard of runs with the exact same privileges in your build and, if it reaches the bundle, in your users’ browsers.

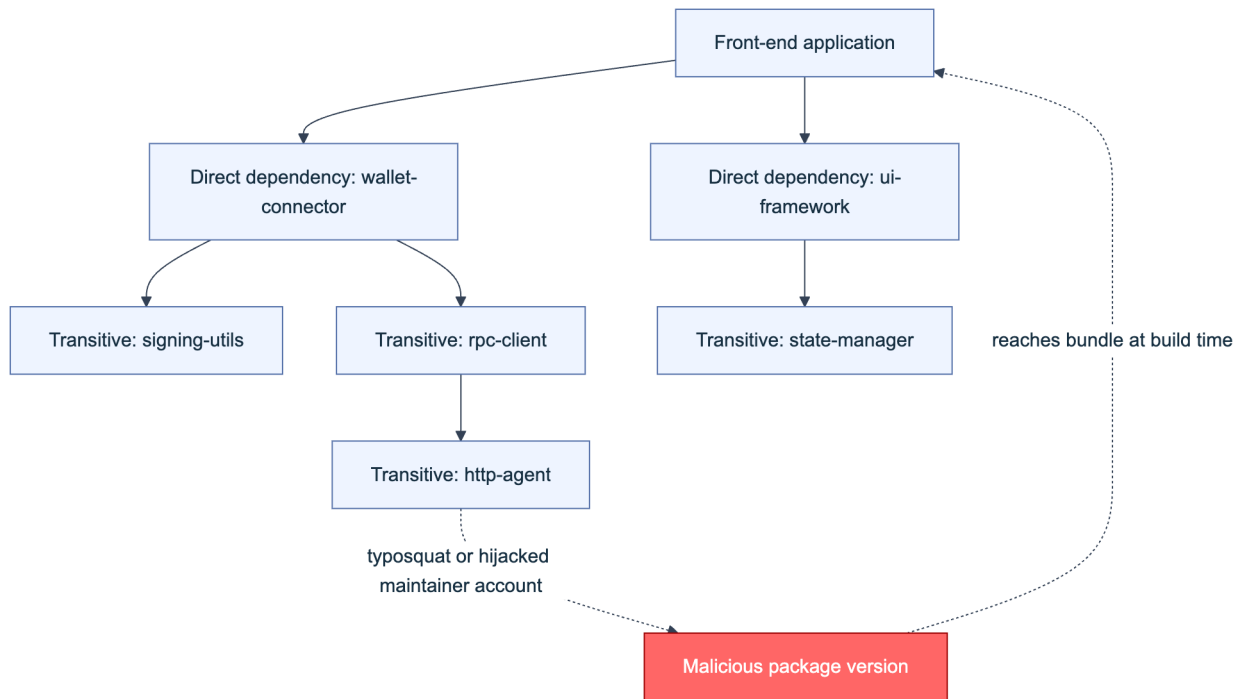


Figure 6. A malicious update three hops down the transitive graph reaches the application with the same authority as a first-party module.

5.2 SBOM (CycloneDX, SPDX 3.0) and VEX

A Software Bill of Materials (SBOM) is a formal, machine-readable inventory of every component in a piece of software, its version, and its known relationships to other components. Without one, “what are we running” is a question only answerable by re-resolving the dependency tree at the moment of the incident, which is too late. Two formats dominate: [CycloneDX](#), an OWASP project ratified as ECMA-424 in June 2024 (1st edition) with version 1.6 adding Cryptographic Bill of Materials support and CycloneDX Attestations, and [SPDX 3.0](#), published by the Linux Foundation in April 2024 as a ground-up model overhaul with dedicated AI and data profiles, building on the earlier ISO/IEC 5962:2021 standardization of SPDX 2.x ([Linux Foundation](#)).

```
# Generate a CycloneDX SBOM from a Node.js project
npx @cyclonedx/cyclonedx-npm --output-file bom.json
```

```
{
  "bomFormat": "CycloneDX",
  "specVersion": "1.6",
  "components": [
    {
      "type": "library",
      "name": "wallet-connector",
      "version": "3.2.1",
      "purl": "pkg:npm/wallet-connector@3.2.1"
    }
  ]
}
```

An SBOM alone tells you what is present, not what matters. Vulnerability Exploitability eXchange (VEX) closes that gap: it is a companion statement, standardized by [CISA's minimum-elements guidance](#), asserting whether a known vulnerability in a listed component is actually reachable and exploitable in your specific build (`affected`, `not_affected`, `fixed`, `under_investigation`). A CVE in a transitive logging library that your build never invokes at the vulnerable code path is `not_affected`; publishing that VEX statement lets downstream consumers and your own triage queue stop re-flagging it every scan. Regulation gives SBOM production a deadline as well as a best practice: U.S. [Executive Order 14028](#) requires SBOMs for software sold to federal agencies, and the EU [Cyber Resilience Act](#) will require manufacturers of products with digital elements sold in the EU to maintain a machine-readable SBOM from December 2027.

EO Section 4 Tasks and Timelines

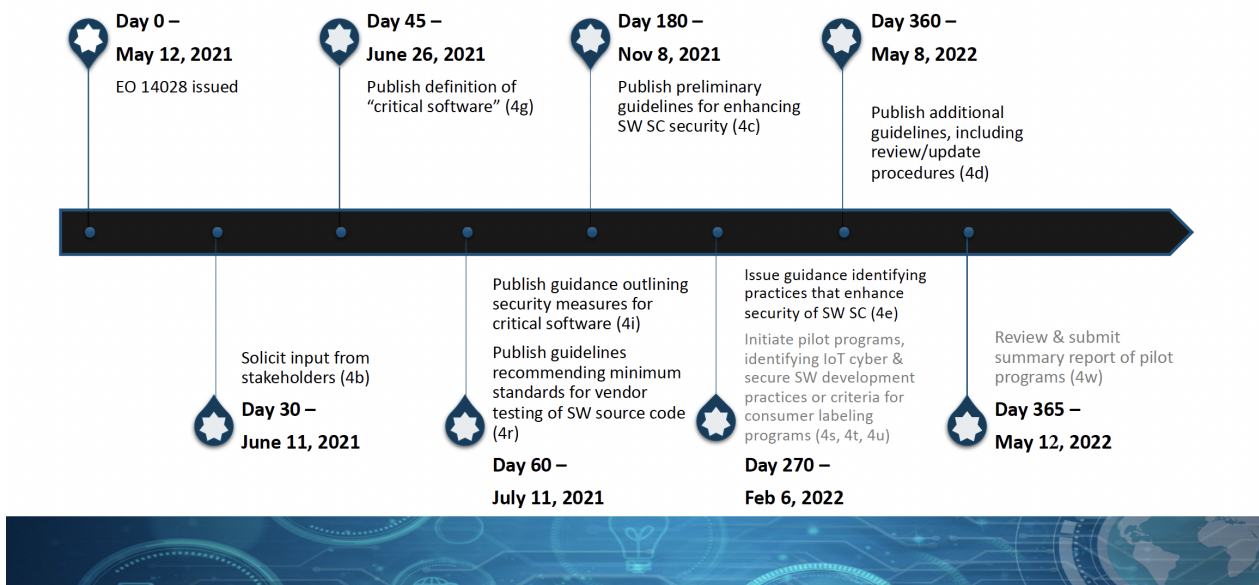


Figure. NIST's own published task-and-timeline chart for Executive Order 14028 Section 4, the software supply chain security mandate that made federal SBOM submission a compliance deadline rather than a voluntary best practice. Source: [NIST](#), public domain (U.S. government work).

5.3 Vulnerability Scanning and Real-Time Monitoring

A point-in-time scan tells you what was vulnerable when you ran it; a dependency added a known-bad version last month and never revisited stays invisible until someone scans again. Treat scanning as a continuous pipeline stage, not a pre-release checklist item. [OSV-Scanner](#), Google's Apache-2.0 CLI, queries the aggregated [OSV.dev](#) database (30-plus upstream sources including GitHub Security Advisories and npm audit data) against your lockfile and, as of v2.0, offers guided remediation and container-layer attribution ([Google Security Blog](#)).

```
osv-scanner --lockfile=package-lock.json
npm audit --audit-level=high
```

Point-in-time scanning must be paired with real-time monitoring that reacts the moment a new advisory publishes against a dependency already in your tree, regardless of when you last ran a scan. GitHub Dependabot alerts and Renovate both watch the GitHub Advisory Database and OSV.dev continuously and can open an automated pull request the same day a CVE lands; commercial tools such as Snyk and Socket.dev add behavioral detection (a package's new version suddenly adding a `postinstall` network call, for instance) that pure CVE matching misses entirely, which matters because the `chalk/debug` and Shai-Hulud campaigns of 2025 shipped malicious code with no CVE assigned for days ([CISA Alert AA25-266A](#)). Route every alert to the on-call channel that also owns the incident response process (Handbook 06) so a Critical dependency alert gets the same urgency as a production page.

5.4 License and Maintenance Health

Two properties beyond “is it vulnerable” determine whether a dependency is safe to keep: its license and its maintenance health. License risk is legal risk that can force a rewrite under deadline pressure if it goes unnoticed until a release audit. Maintenance health is a leading indicator of security risk: an unmaintained package accumulates unpatched vulnerabilities and is a far easier maintainer-account takeover target than one with an active team and enabled two-factor authentication.

Signal	Low risk	Investigate	High risk
License	MIT, Apache-2.0, BSD (permissive)	LGPL, MPL (weak copyleft, review linking)	GPL/AGPL in a proprietary bundle, or no declared license
Last release	Under 6 months	6-18 months	Over 18 months, or archived repository
Maintainer count	3+ active	1-2 active	Single maintainer, no succession plan
OpenSSF Scorecard	7-10	4-6	0-3, or unscored
Open critical issues	Triaged, low count	Growing backlog	Long-standing unaddressed reports

Scorecard runs 18 automated checks (branch protection, signed releases, dangerous workflow patterns, and a `Maintained` heuristic among them) and publishes a 0-10 score per project, with weekly runs across roughly a million critical open-source projects logged to a public BigQuery dataset ([OpenSSF](#)). Tools such as FOSSA, ScanCode Toolkit, and ClearlyDefined automate license detection across a full tree; run one in CI rather than trusting a `package.json` license field, which is frequently wrong or absent.

5.5 Dependency Update and Patching Policy

Awareness without a policy just produces a longer list of things nobody acts on. A patching policy converts scan output into a committed SLA (service level agreement) by severity, and it should live in the same repository as the code it governs so it is versioned and enforceable in review.

Severity (CVSS)	Action window	Mechanism
Critical (9.0-10.0)	Patch or mitigate within 24 hours	Emergency PR, bypass normal release cadence
High (7.0-8.9)	Patch within 7 days	Automated Dependabot/Renovate PR, expedited review
Medium (4.0-6.9)	Patch within 30 days	Batched into next scheduled dependency sweep
Low (0.1-3.9)	Patch at next minor release	Standard dependency bump cycle

Weight the window with exploit-prediction signal, not CVSS alone: EPSS (Exploit Prediction Scoring System) scores a CVE's probability of near-term exploitation, and a Medium-severity finding with a high EPSS score deserves the Critical lane. Automate the mechanical part: Dependabot or Renovate opens the PR, CI runs the full test suite and a fresh SBOM diff, and a human reviews only the diff and the changelog rather than re-deriving the update from scratch. Pin exact versions in the lockfile (never a floating range in the manifest that CI resolves fresh each run) so an update is always a reviewed, atomic, revertible commit, never a surprise picked up silently on the next `npm install`.

5.6 SLSA and Provenance for Build Artifacts

An SBOM says what is in an artifact. Provenance says how that artifact came to exist: which source commit, which build system, which build steps, run by which identity. [SLSA](#) (Supply-chain Levels for Software Artifacts), an OpenSSF-incubated framework that reached v1.0 in April 2023 and has since added a source track alongside its original build track in v1.2, defines four levels of build integrity assurance:

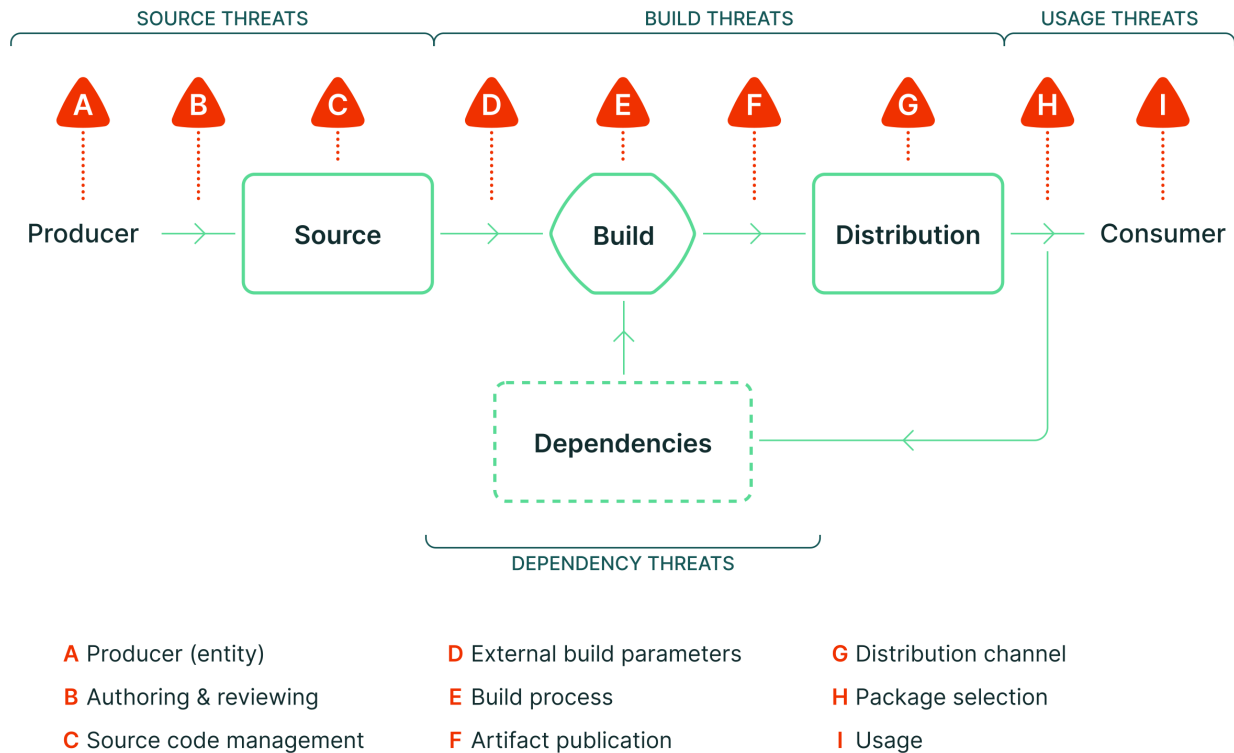


Figure. SLSA's threat model locates nine distinct compromise points across the producer-to-consumer path. It makes the key operational point visually: signing the final package does not address source tampering, a compromised build, a poisoned distribution channel, or unsafe dependency selection. Source: *SLSA v1.2, Threats and mitigations*, Community Specification License 1.0.

Level	Requirement	Forging cost
L0	No provenance	Trivial
L1	Provenance exists, describing build platform and process	Unsigned; trivial to forge
L2	Provenance signed by a hosted build platform	Requires an explicit attack, not a config mistake
L3	Build platform is hardened and isolated; builds cannot access each other's state or the signing key	Requires compromising the build platform itself

([SLSA v1.2 Build requirements](#); [SLSA v1.2 Source requirements](#).) Hosted CI and OIDC-backed provenance make L2 or L3 feasible, but using a hosted runner does not itself confer a level. The selected builder must have a documented SLSA platform assessment and the consuming policy must verify its identity, provenance subject, source, and expected workflow. npm CLI 11.5.1 and later can emit provenance attestations for trusted-published packages ([npm docs](#)); Chapter 6 covers generating and verifying these attestations end to end. Assess the Source Track separately, because high build assurance does not stop an authorized maintainer from merging malicious source unilaterally.

6. Build and Pipeline Security

Everything Chapter 5 inventories still has to pass through a build. The pipeline that compiles, bundles, and publishes your front end holds more concentrated power than almost any other system you operate: a single injected line in a build script reaches every user on the next deploy, no dependency compromise required.

6.1 Build Reproducibility

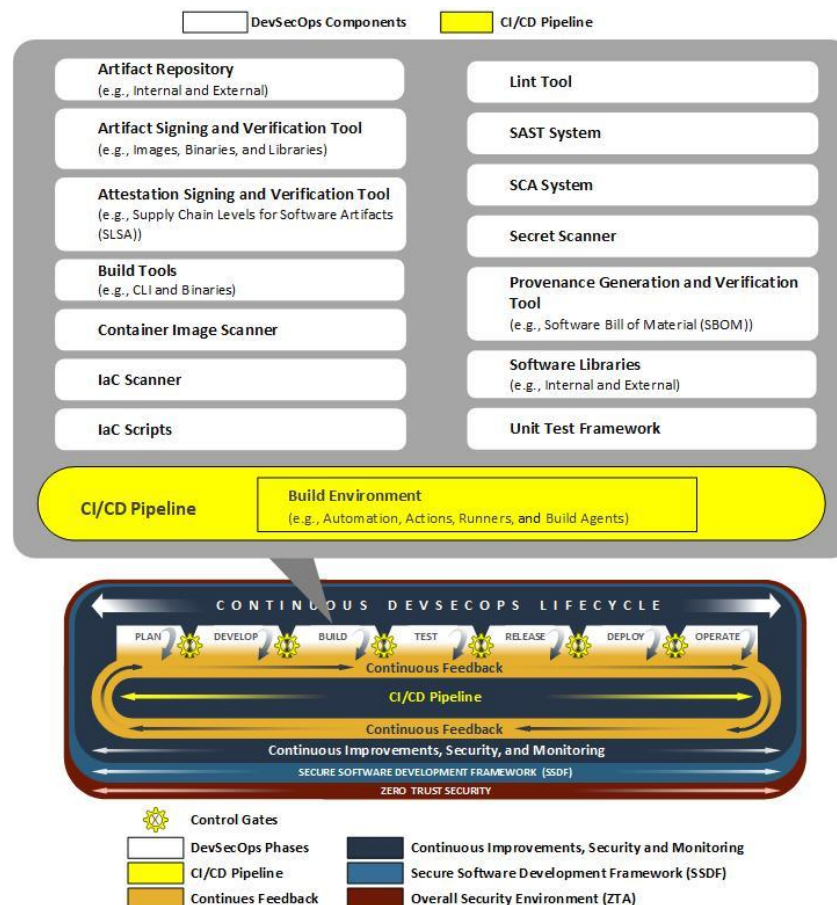


Figure. NIST's build-phase model makes the build boundary explicit: source, dependencies, configuration, and toolchain enter a controlled build service, while artifacts, metadata, logs, and integrity evidence leave it. Reproducibility is meaningful only when those inputs are pinned and the output evidence is retained. Source: *NIST NCCoE, Build Phase*, public domain (U.S. government work).

A build is reproducible when, given the same source, the same build environment, and the same instructions, any party can independently produce a bit-for-bit identical artifact ([Reproducible Builds project](#); the effort originated inside Debian in 2013 and by 2017 had made over 90% of Debian's archive reproducible, [Debian Wiki](#)). Reproducibility is what makes provenance verifiable rather than merely asserted: SLSA provenance says "this is what the build platform claims it did," and reproducibility lets a third party rebuild from the same source and confirm the claim independently, closing the trust gap a compromised or misconfigured build platform would otherwise leave open.

Common sources of non-determinism defeat this quietly: embedded build timestamps, non-deterministic file iteration order when bundling, absolute file-system paths baked into source maps, and floating compiler or bundler versions resolved fresh on every run. Pin your toolchain (exact Node.js version, exact bundler version, a committed lockfile), strip or normalize timestamps, and sort file lists before bundling. Verify with `diff` after two independent builds, or with `diffoscope` for a structured, recursive diff when the outputs are not text:

```
# Build twice in isolated environments, then diff
diffoscope build-a/app.bundle.js build-b/app.bundle.js
```

A clean `diffoscope` output between two builds run a week apart, on different machines, is strong independent evidence that no build-time tampering occurred between them.

6.2 Secrets and Credentials in Build

The build runner is, by necessity, one of the most privileged systems in the pipeline: it holds the credentials to publish packages, deploy to the CDN, and often to read source across the whole organization. The [tj-actions/changed-files compromise \(CVE-2025-30066\)](#) covered in Part I dumped runner memory into public workflow logs specifically to harvest whatever secrets were resident at build time, across more than 23,000 repositories ([CISA](#)). Long-lived static credentials sitting in CI secret stores are exactly what that class of attack is built to steal.

The structural fix is to stop having long-lived secrets to steal. OpenID Connect (OIDC) issues a short-lived, workflow-scoped token at build time that expires in minutes and cannot be replayed outside the run that requested it; npm's trusted-publishing feature, generally available since July 2025, lets a package accept publishes only from a named GitHub Actions or GitLab CI/CD workflow via OIDC, eliminating the static `NPM_TOKEN` entirely ([GitHub Changelog](#)).

Credential type	Lifetime	Exfiltration value	Replay outside the run
Static npm/PyPI token in CI secret	Indefinite until rotated	High: full publish rights	Yes
Long-lived cloud IAM key	Indefinite until rotated	High: full account scope	Yes
OIDC-issued short-lived token	Minutes	Low: expires before most attackers can act	No

Where static secrets remain unavoidable, scope them to least privilege, run `gitleaks` or `trufflehog` in CI to catch anything committed by accident, and never let install scripts run unaudited: `npm install --ignore-scripts` blocks the postinstall hook class of attack that turned a compromised Axios-adjacent package into a remote-access-trojan dropper, at the cost of breaking legitimate build-time hooks like `node-gyp` or `sharp`, which is why teams move to an explicit allowlist rather than a blanket flag ([nodejs-security.com](#)).

6.3 Artifact Signing and Verification

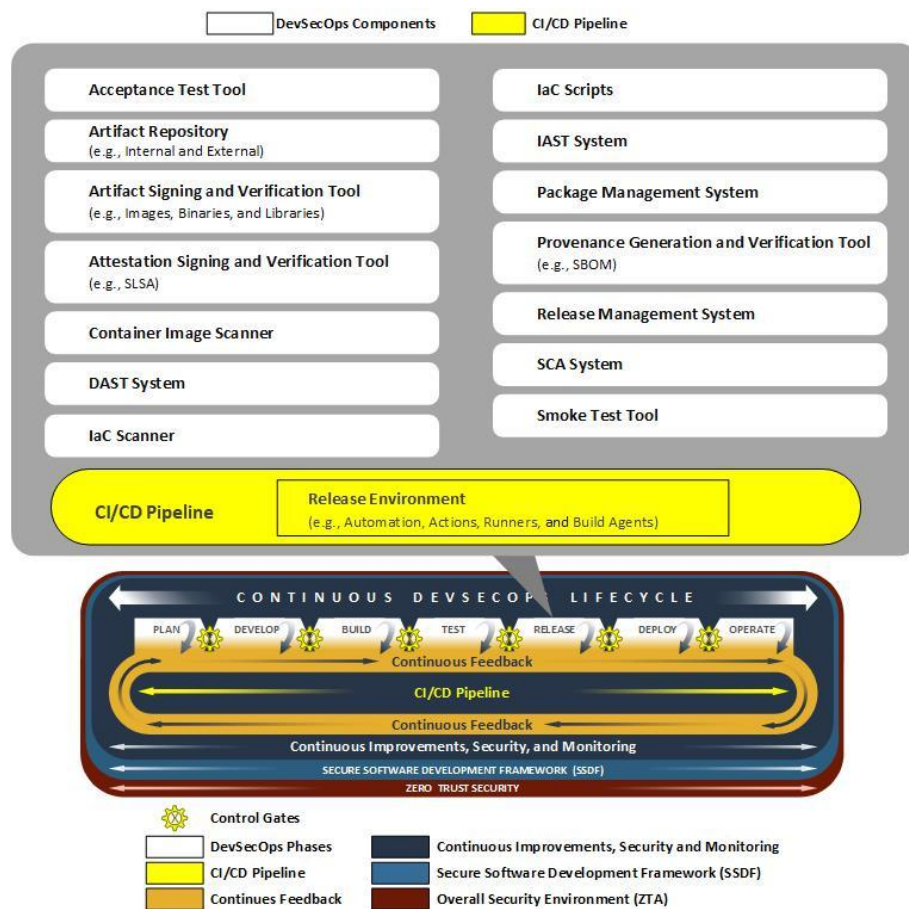


Figure. NIST's release phase separates producing an artifact from authorizing it for use. Signature verification, provenance policy, and an immutable promotion decision belong at this gate; copying an unsigned build output directly to production collapses the build and release trust boundaries. Source: [NIST NCCoE, Release Phase](#), public domain (U.S. government work).

Part II, Section 4.3 introduced Sigstore keyless signing for published assets; here it closes the loop from source to deploy. Every artifact your build produces, the bundled JavaScript, the SBOM itself, and the SLSA provenance attestation, should be signed in CI before it ever reaches an artifact registry, and every one of those signatures should be checked again immediately before promotion to the CDN. This turns “the deploy step trusts the build step” into “the deploy step verifies the build step,” which is the difference between a compromised runner silently shipping to users and a compromised runner getting caught at the gate.

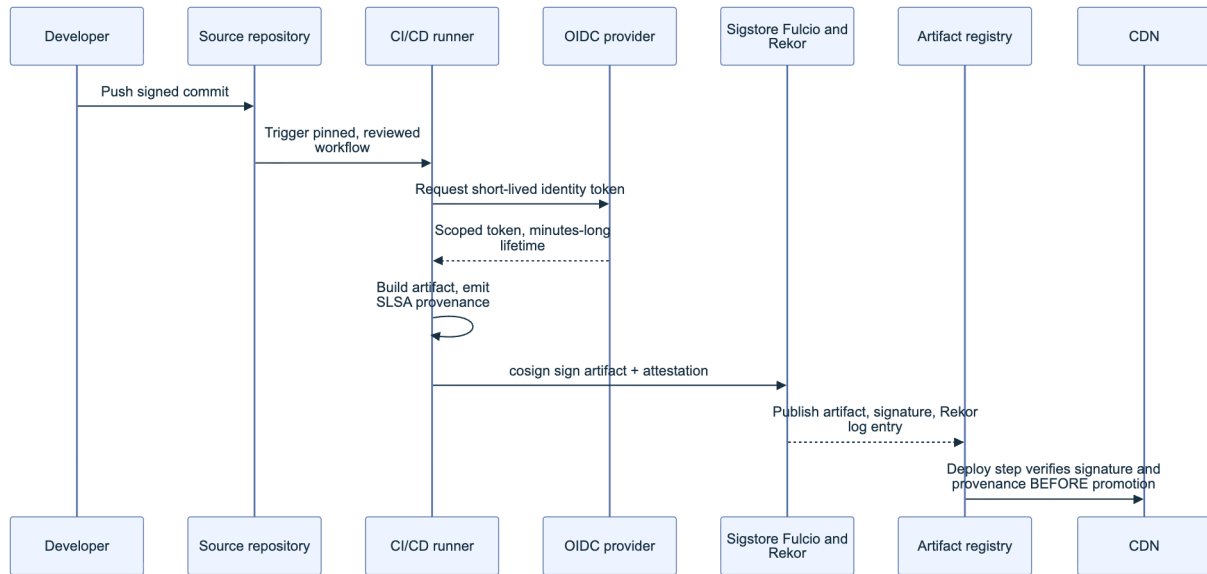


Figure 7. Build-to-deploy signing and verification. A break in the chain anywhere upstream of the deploy gate is caught there, not discovered by a user.

```
# Verify a signed artifact and its SLSA provenance before promoting it
cosign verify-attestation --type slsaprovenance \
  --certificate-identity-regexp "https://github.com/org/repo/.*" \
  --certificate-oidc-issuer https://token.actions.githubusercontent.com \
  app.bundle.js
```

The `slsa-verifier` tool wraps this check specifically for SLSA-formatted provenance and is the reference implementation to script into a deploy gate as a hard failure, not a warning.

6.4 CI/CD Hardening

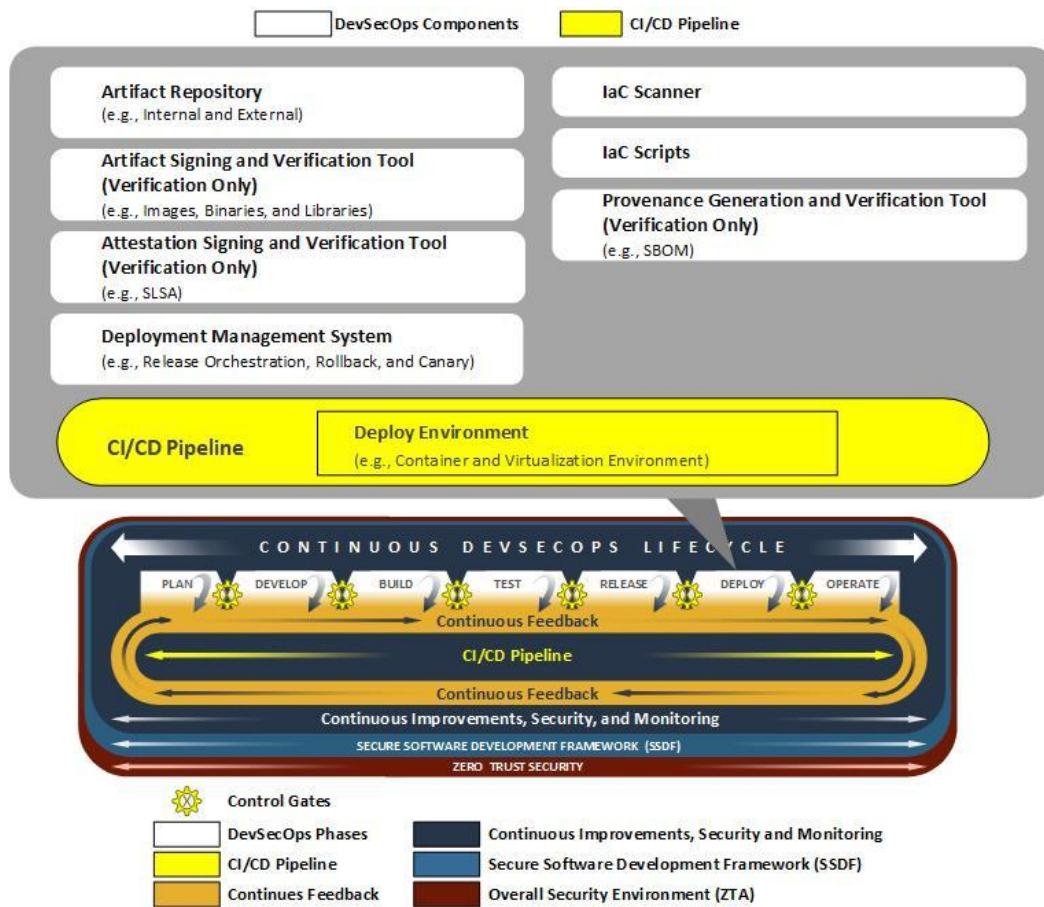


Figure. NIST's deploy-phase model shows why deployment automation needs its own identity and policy boundary. The deployer should accept only an authorized release, record exactly what reached each environment, and expose no general-purpose path from a compromised build runner into production. Source: *NIST NCCoE, Deploy Phase*, public domain (U.S. government work).

Everything above assumes the pipeline itself has not already been widened into an easy target. Harden it as its own system, not as an afterthought to the code it builds.

Control	Why it matters
Pin every third-party action to a full commit SHA, never a mutable tag	Tags are re-pointable by the action's maintainer or a compromised maintainer account; a SHA is immutable (GitHub Secure Use)
Set <code>permissions:</code> to least privilege at the workflow and job level	Default <code>GITHUB_TOKEN</code> scope is broader than most jobs need; an injected step inherits whatever scope you grant
Avoid <code>pull_request_target</code> with checkout of untrusted head refs	Runs with write-scoped secrets against attacker-controlled code from a fork
Require reviewed status checks and branch protection on the default branch	Prevents a single compromised or careless push from reaching the build directly
Use ephemeral, single-job runners over persistent self-hosted runners for untrusted workloads	A persistent runner accumulates state and secrets across jobs that a single compromised job can then read

Control	Why it matters
Enable org-level policy to block unpinned or unapproved actions	GitHub Actions policy has supported blocking and enforcing SHA pinning at the organization level since August 2025 (GitHub Changelog)
Run a dependency-review or OSV-Scanner action on every PR	Catches a newly introduced vulnerable or malicious dependency before merge, not after

```
# .github/workflows/build.yml (excerpt)
permissions:
  contents: read
  id-token: write # OIDC only; no static publish token in scope
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@de0fac2e4500dabe0009e67214ff5f5447ce83dd # v6.0.2
      - run: npm ci --ignore-scripts
      - run: npm run build
```

Treat the workflow file with the same review rigor as production code: it is production code, and it runs with more ambient authority than most of what it builds.

7. Third-Party Scripts and Widgets

The pipeline can be flawless and the page still loads code you did not build. Analytics, chat widgets, A/B testing, error monitoring, and ad tags are near-universal on production front ends, and every one of them is a script that executes with the same DOM access and origin privileges as your own code.

7.1 Risks of Inline and External Scripts

Any script that runs in your page's origin can read and rewrite the DOM, including the swap form, the recipient field, and the “confirm” button a user is about to click through to their wallet. It does not need a vulnerability in your code to do this; it needs only to be present. This is the same elevation-of-privilege path the [threat model in Part I, Section 2.3](#) walks through: a third-party script's compromise is not contained to “their” functionality, it is a compromise of your entire page.

The [British Airways breach of 2018](#) is the canonical non-Web3 case: attackers modified a third-party JavaScript library (a fork of Modernizr) loaded on the payment page, which silently skimmed card data from roughly 500,000 customers over two weeks before detection. The UK Information Commissioner's Office initially proposed a fine of £183.39 million and settled at £20 million in October 2020, one of the largest data-protection penalties issued under GDPR to that point. Nothing about British Airways' own code was defective; the third-party script it trusted was.

Inline scripts (code written directly into the HTML rather than fetched from a `src` URL) are structurally worse from an integrity standpoint: SRI, covered in Part II Section 4.1, cannot pin an inline block at all, and the only defense available is a CSP nonce or hash that changes every response. Categorize every script your page loads by what it can reach: an analytics snippet on a marketing page is low stakes, the same snippet loaded on the transaction-signing page is not, regardless of how reputable the vendor is.

7.2 CSP and Permissions

Part II Section 4.5 introduced a nonce-based Content Security Policy (CSP) with `strict-dynamic` as the baseline. Applied specifically to third-party scripts, two directives matter most: `script-src` (or the nonce plus `strict-dynamic` propagation) controls what may execute at all, and `connect-src` controls where a script that does execute may send data, which is the directive that actually stops a compromised analytics tag from exfiltrating a signed transaction payload to an attacker's collection endpoint.

```
Content-Security-Policy:
  script-src 'nonce-r4nd0m' 'strict-dynamic';
  connect-src 'self' https://analytics.trusted-vendor.example;
  frame-src https://chat-widget.trusted-vendor.example;
  object-src 'none';

Permissions-Policy:
  clipboard-write=(self),
  usb=(),
  serial=(),
  hid=()
```

The `Permissions-Policy` header (successor to the older Feature-Policy) is CSP's complement for browser capabilities rather than resource loading: it can deny a third-party iframe access to the clipboard, USB, serial, and HID APIs that hardware wallet browser extensions rely on, so an embedded widget cannot silently intercept a WebHID or WebUSB signing flow it has no legitimate reason to touch. Scope both headers per route where practical: the marketing site can afford a looser `connect-src` than the page that renders a signing prompt.

7.3 Auditing and Minimizing Third-Party Use

The most reliable third-party script is the one you removed. Every vendor script is a standing grant of code execution that a security review has to re-justify, not a one-time integration decision. Build and maintain a live inventory: script origin, purpose, owner, last review date, and whether it loads on any signing-adjacent route.

Vendor tier	Examples	Placement rule
Tier 1: touches signing flow or wallet APIs	Wallet connector SDKs, transaction simulators	First-party review of source, pinned version, SRI mandatory, no floating CDN reference

Vendor tier	Examples	Placement rule
Tier 2: same-origin, no signing adjacency	Analytics, error monitoring, feature flags	SRI where the vendor supports it, <code>connect-src</code> restricted to the vendor's documented endpoints
Tier 3: cosmetic or marketing only	Chat widgets, ad tags, A/B test snippets	Sandboxed iframe with <code>allow</code> attributes minimized, never on transaction-signing routes

Tag managers deserve specific scrutiny: a tool like Google Tag Manager (GTM) is itself a supply-chain multiplier, because it lets non-engineering stakeholders push arbitrary new scripts to production outside code review, which is exactly the mechanism Magecart-family skimmers have repeatedly abused to reach payment pages without touching the target's own repository. If GTM or an equivalent is in use on any route that leads toward a wallet interaction, require the same review gate for tag changes that a code change would get.

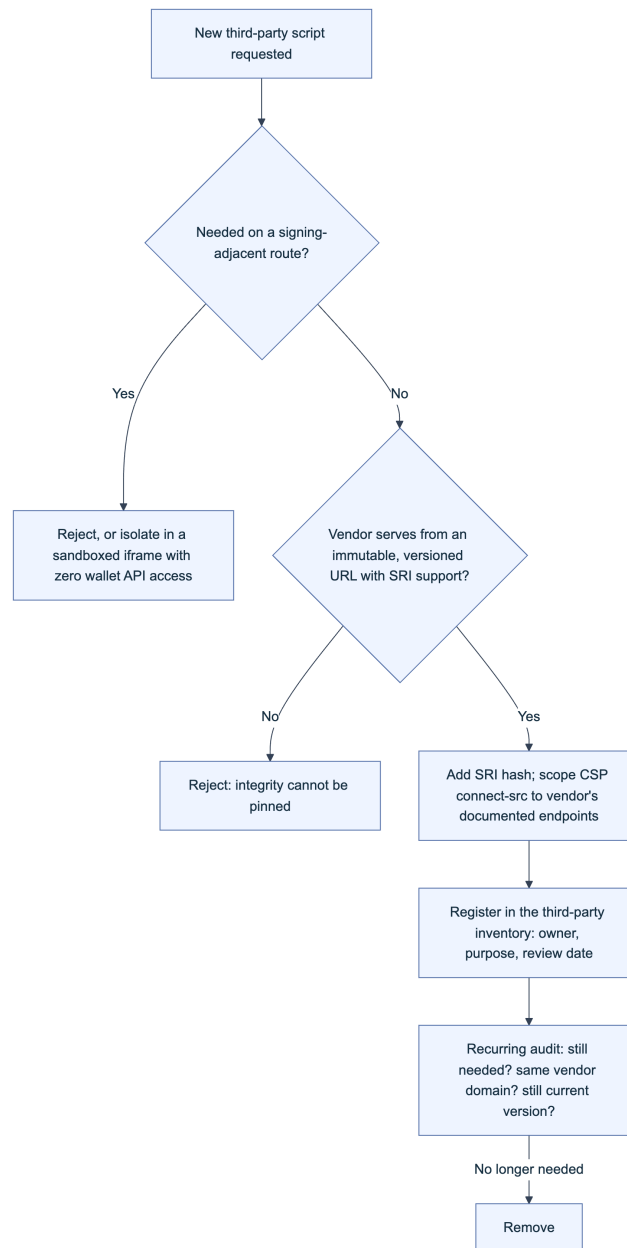


Figure 8. Onboarding gate for a new third-party script. The default answer is no; every “yes” becomes a tracked, re-auditable liability.

Set a recurring cadence, quarterly is reasonable for most teams, to re-run this audit against every script already in production, not only new requests. Vendors get acquired, domains lapse and get re-registered, and a script that was safe at onboarding time does not stay safe by default; the [Part I observation](#) that trust granted once and never rechecked is the recurring failure mode applies with equal force to a third-party script tag as it does to a CDN host.

Key controls for Part III

- Generate a CycloneDX or SPDX SBOM on every build and pair it with VEX statements so vulnerability triage reflects actual reachability, not raw CVE counts.

- Run continuous vulnerability monitoring (Dependabot, Renovate, OSV-Scanner) rather than point-in-time scans, and enforce a written patch SLA by severity.
- Target SLSA Build Level 3 by using hosted runners with OIDC-issued provenance and short-lived credentials in place of static publish tokens.
- Sign every build artifact and its provenance attestation, and make verification a hard gate before CDN promotion, not a warning.
- Pin every CI/CD action to a commit SHA, scope `GITHUB_TOKEN` permissions to the minimum a job needs, and block unpinned actions at the organization level.
- Inventory every third-party script by vendor tier, keep signing-adjacent routes free of Tier 2 and Tier 3 scripts entirely, and re-audit the inventory on a fixed schedule.

Part IV: Threat Models by Component

Parts II and III gave controls. This part gives structured threat models for each component of the delivery stack, so a team can reason about its own architecture rather than apply controls by rote. Each model names the assets, the trust boundaries, the adversary's likely moves in STRIDE terms, and the controls that cut each branch. The chapter on common front-end vulnerabilities and the closing tools chapter turn the models into things you can test and buy.

8. Threat Model: CDN

The CDN is the closest untrusted party to the user. Its assets, from a defender's view, are the integrity of served content, the confidentiality of any origin secrets it holds, and its availability. The trust boundary is the CDN control plane (accounts, API tokens, edge configuration) plus the edge-to-user and edge-to-origin transport.

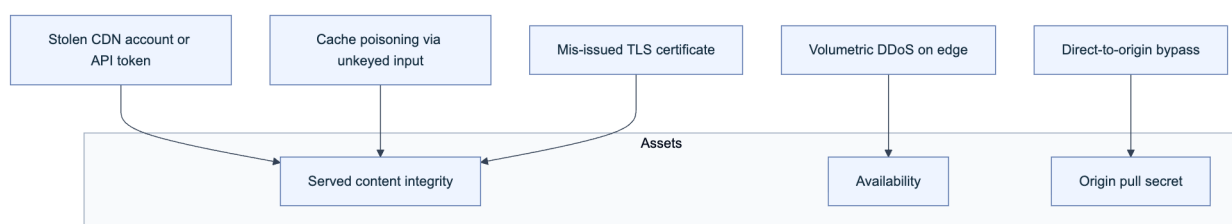


Figure 1. CDN threat model. Most paths to loss of content integrity route through the control plane or the cache, which is why account hardening and SRI dominate the control set.

Asset	STRIDE	Threat	Control
Content integrity	Tampering, Spoofing	Attacker with CDN account substitutes served JS	SRI pins on embedding pages; immutable versioned assets; hardware MFA on the account
Content integrity	Tampering	Cache poisoning via unkeyed header	Normalize request headers; key on Vary ; SRI as backstop

Asset	STRIDE	Threat	Control
Content integrity	Spoofing	Mis-issued certificate for your host	CAA records; Certificate Transparency monitoring
Availability	Denial of service	Volumetric or application-layer DDoS	Anycast scrubbing, rate limits, multi-CDN or static fallback
Origin secret	Information disclosure	Attacker reaches origin directly, bypassing edge	Authenticated origin pull, IP allowlist, mutual TLS

The residual risk after these controls is a full CDN control-plane compromise that also strips your SRI, which is only possible if the attacker also controls the HTML your origin serves. That is why the origin and the CDN should never share a single credential or blast radius.

9. Threat Model: Front-End Application Stack

The front-end application is your first-party code plus the way it composes third-party code at runtime. The asset is the integrity of the transaction the user ultimately signs. The dominant threat is any injection that lets attacker-controlled script run in your origin, because such script can read and rewrite the DOM of a signing form.

The trust boundaries inside the application are easy to miss. A component that renders a token name, an ENS record, or an NFT metadata field is trusting data that an attacker may control. A `dangerouslySetInnerHTML`, an unsanitized template, or a vulnerable dependency that pollutes `Object.prototype` all cross the boundary between untrusted data and executable behavior. Model each rendering path as a question: if this value were attacker-chosen, what could it do to the signing flow?

The controls are layered. A nonce-based CSP with `strict-dynamic` (Part II) denies injected scripts the ability to execute. Trusted Types close the DOM injection sinks. Output encoding and a hardened templating layer prevent the injection in the first place. Critically, the design should isolate the signing path: the code that constructs the recipient, amount, and calldata should not render untrusted content, and the wallet request should be built from validated, typed values rather than from strings assembled near user-controlled DOM.

10. Common Front-End Vulnerabilities (Web and Mobile)

The web weaknesses that matter most in Web3 are the ones that reach the signing surface. This chapter enumerates them defensively so they can be tested for.

Cross-site scripting (XSS) remains the highest-impact web weakness here. Reflected and stored XSS let an attacker run script in your origin; DOM-based XSS does the same through client-side sinks. In a wallet context the payoff is not a stolen cookie, it is a rewritten transaction. Defenses are the OWASP [XSS prevention](#) staples: context-aware output encoding, a strict CSP, and Trusted Types.

Prototype pollution lets an attacker set properties on `Object.prototype` through a crafted key path, changing the behavior of code that later reads those properties. In a bundler-heavy front end it can escalate to XSS or logic bypass. Freeze critical objects, validate keys, and prefer `Map` over plain-object dictionaries for untrusted keys.

Clickjacking and UI redress frame your dApp inside a hostile page to trick users into approving actions. Defend with `frame-ancestors 'none'` in CSP and `X-Frame-Options`.

Open redirects and postMessage abuse matter because wallets and connectors communicate across frames. Validate every `postMessage` origin against an allowlist and never trust `event.data` shape without checking it.

Weakness	Web3-specific impact	Primary control
XSS (reflected, stored, DOM)	Transaction rewrite before signing	Encoding, strict CSP, Trusted Types
Prototype pollution	Logic bypass, escalation to XSS	Key validation, frozen objects
Clickjacking	Coerced approval	<code>frame-ancestors 'none'</code>
<code>postMessage</code> abuse	Malicious connector messages	Origin allowlist, schema checks
Insecure storage of session or keys	Theft of session or, worse, key material	Never store secrets in <code>localStorage</code> ; use platform keystores

On mobile, the same weaknesses appear inside WebViews, plus platform issues covered in Chapter 14.

11. Threat Model: NPM/Package Ecosystem

The package ecosystem is the attacker's preferred entry because one compromised package reaches every downstream project. The assets are the integrity of installed code and the confidentiality of any secrets present at install time. The trust boundary is every direct and transitive dependency and every maintainer account behind them.

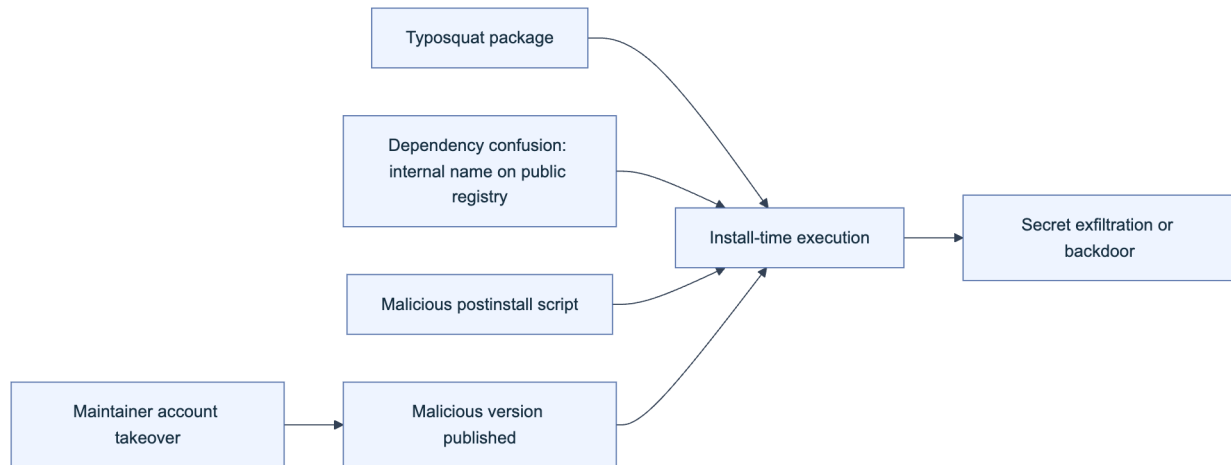


Figure 2. Package ecosystem threat model. The recurring pattern is code execution at install time, which is why lockfiles, `--ignore-scripts`, and provenance verification are the anchor controls.

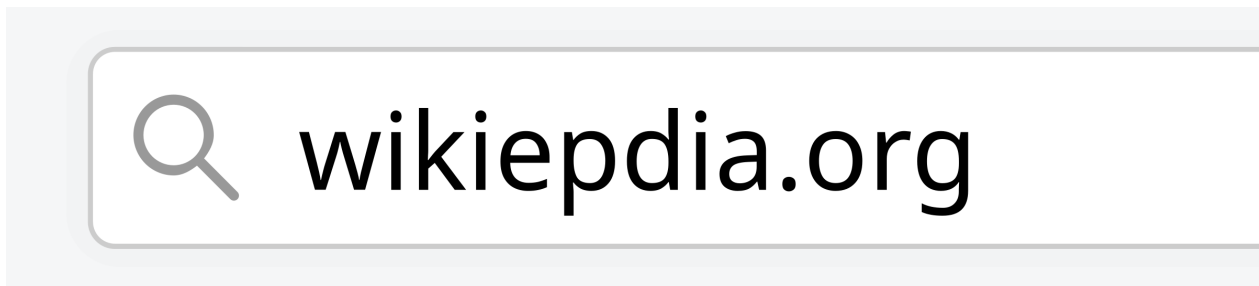


Figure. Typosquatting works the same way whether the misspelled target is a domain in a browser address bar or a package name in an install command: a small, plausible spelling variation is enough to redirect a trusting user or a trusting build to the attacker's resource instead of the intended one. Source: [Wikimedia Commons](#), CCO.

The named cases make the model concrete. The [Ledger Connect Kit compromise](#) was a maintainer-account takeover via a phished former employee. Dependency confusion exploits a resolver that prefers a public package over your intended internal one when names collide. The controls: pin exact versions with a committed lockfile, scope internal packages and configure the registry to prevent public override, disable install scripts by default (`npm install --ignore-scripts` and audited allowlists), verify provenance where publishers provide it, and review the dependency delta on every update rather than trusting the version bump. Part III covers the SBOM and SLSA mechanics that make this auditable.

12. Threat Model: Build and Deploy Pipeline

The pipeline turns source into the artifact users run, and it usually holds the most powerful secrets in the organization. The asset is the integrity of the produced artifact and the confidentiality of signing keys and deployment credentials. The trust boundary is the CI runner, every third-party build action, and the artifact and deploy targets.

The [tj-actions/changed-files compromise of March 2025 \(CVE-2025-30066\)](#) is the reference incident: a popular GitHub Action had its version tags repointed to a commit that dumped the runner's memory, spilling CI secrets into logs across more than 23,000 repositories. The model's lesson is that a mutable reference to third-party build code is a standing compromise waiting to happen.

Asset	STRIDE	Threat	Control
Artifact integrity	Tampering	Compromised third-party action injects code	Pin actions to a full commit SHA, never a tag; review action source
Signing keys	Information disclosure	Secret dumped from runner memory or logs	Least-privilege, short-lived OIDC tokens; no long-lived secrets in CI; log scanning
Artifact integrity	Repudiation	No record of who built what	Signed provenance (SLSA), Sigstore transparency log
Deploy target	Elevation of privilege	CI token over-scoped to production	Separate build and deploy identities; environment protection rules

Pin third-party actions to an immutable commit SHA, isolate runs so one cannot influence another, keep signing material out of user-defined build steps (the SLSA Level 3 requirement), and generate signed provenance so a downstream verifier can reject an artifact that did not come from your pipeline.

13. Threat Model: Runtime (Browser, Mobile)

At runtime the code finally executes in an environment you do not control: the user's browser or mobile app, alongside extensions, other tabs, and any third-party script the page loaded. The asset is the integrity of the transaction and the confidentiality of any in-memory secrets. The threats are malicious or vulnerable browser extensions that can read and modify page content, other-origin scripts that escape their intended sandbox, and a compromised wallet or connector.

Defenses acknowledge that the runtime is hostile by default. Minimize the third-party scripts that run in your origin (Chapter 7), use CSP and Trusted Types to contain what does run, and treat the wallet's own confirmation screen as the last line of defense: design flows so the user verifies the critical transaction details on a trusted surface, ideally a hardware wallet screen, rather than trusting the web page's rendering. The UX Security Handbook (09) covers the signing surface in depth.

14. Mobile Application Security (Web3 Front-Ends)

Mobile Web3 front ends are either native apps embedding a WebView or fully native apps calling the same contracts. Both inherit the web weaknesses inside any WebView and add platform-specific concerns: insecure local storage of session tokens or key material, weak certificate validation that enables interception, deep-link and intent hijacking that can spoof in-app flows, and reverse-engineering of shipped binaries. Align testing with the [OWASP Mobile Application Security \(MASVS/MASTG\)](#) project. The anchor controls: store secrets only in the platform keystore (iOS Keychain, Android Keystore), pin or strictly validate TLS, validate deep-link parameters as untrusted input, and harden the WebView (disable unnecessary JavaScript bridges, load only your own origin). The SDK Security Handbook (08) covers mobile SDK testing.

15. Security Tools and Resources for Front-End

Tools operationalize the models above. Group them by the level they defend.

Level	Purpose	Representative tooling
Code	Dependency and SCA scanning	npm audit, OWASP Dependency-Check, OSV-Scanner, Socket
Code	SBOM generation	CycloneDX generators, Syft
Build	Provenance and signing	SLSA generators, Sigstore Cosign
Build	Secret scanning	Gitleaks, TruffleHog, native CI secret scanning
Deploy	Header and CSP validation	Mozilla Observatory, CSP Evaluator
Runtime	Client-side monitoring	CSP violation reporting, script inventory and integrity monitoring
Runtime	Certificate monitoring	Certificate Transparency log monitors

Adopt tools where they enforce a control from the models, not for coverage theater. A dependency scanner is only useful with a policy that fails the build on a disallowed finding, and CSP reporting is only useful when a violation raises an alert that a human triages. Part V turns these into a monitoring and response program.

Key controls for Part IV

- Threat-model each component against STRIDE; do not apply controls by intuition.
- Treat any path that renders untrusted data near the signing flow as a first-tier risk.
- Pin third-party build actions to commit SHAs and keep signing material out of build steps.
- Assume the runtime is hostile; verify critical transaction details on a trusted surface.
- Adopt tools only where they enforce a specific control, with a policy that acts on findings.

Part V: Mitigations and Operations

Every control described in Parts II through IV only pays off if it is run consistently, watched continuously, and rehearsed before it is needed. This part turns that theory into operating practice: a checklist a release engineer can lift directly into a runbook, a monitoring layer that converts a silent compromise into a page, and an incident response plan written to be read before an incident rather than during one. Treat it as the closing loop of the handbook, prevention feeds monitoring, monitoring feeds response, and every response feeds a new line back into the checklist.

16. Checklists and Controls

A checklist is only as good as its coverage across the supply chain. This chapter compiles the individual controls from Parts II through IV into artifacts meant for direct use: a control baseline mapped to the four supply chain levels, a release gate, a cadence table for the controls that must recur, and a maturity model for judging how far along a team actually is.

16.1 Control Baseline by Supply Chain Level

Part I introduced four supply chain levels: code, build, deploy, and runtime. Every control this handbook has described attaches to exactly one of them, and a security program with a gap at any level is only as strong as that gap, no amount of hardening at the deploy level compensates for an unsigned, unverified build. The table below is the consolidated baseline: read it as the minimum bar, not the ceiling.

Level	Baseline control	Verification method	Typical owner
Code	Branch protection, signed commits, dependency review on every PR	CI check, required review	Application engineering
Code	SBOM generated for direct and transitive dependencies (Part III, §5.2)	SBOM diff against previous release	Release engineering
Build	Pinned actions/runners, no floating tags, secrets scoped and short-lived (Part III, §6.2-6.4)	CI/CD configuration audit	DevOps / platform
Build	Artifacts signed with SLSA provenance (Part III, §5.6)	<code>slsa-verifier</code> or <code>cosign verify</code> at deploy gate	Security engineering
Deploy	Third-party scripts and styles pinned with SRI, served from immutable URLs (Part II, §4.1, §4.4)	Automated SRI hash audit	Front-end team
Deploy	CDN and registry accounts on hardware MFA, scoped tokens (Part II, §3.1)	Quarterly access review	Platform lead
Runtime	CSP with <code>strict-dynamic</code> , Trusted Types, violation reporting (Part II, §4.5)	Report-only rollout, then enforced	Front-end / AppSec
Runtime	Third-party script inventory reviewed and minimized (Part III, §7)	Quarterly inventory audit	Front-end + security

Mapping controls this way also gives incident response (Chapter 18) a fast triage question: given the entry point of a compromise, which level failed, and which row in this table should have caught it? A gap found during a postmortem should always resolve to adding or fixing a specific row, not a vague instruction to “be more careful.”

16.2 Release Gate Checklist

The release gate is the single point where every prior control is enforced mechanically before an artifact reaches users. Treat a failed checklist item as a hard stop, not a note for later, a release that ships with a known gap is a release that has already accepted the corresponding risk without a decision-maker signing off on it.

Front-End Release Gate Checklist

- ☐ SBOM generated for this build (CycloneDX or SPDX) and diffed against the previous release
- ☐ All new or changed dependencies passed vulnerability and malicious-package scanning (OSV-Scanner / Socket / Phylum) with zero unresolved Critical or High findings
- ☐ Build artifact signed (Sigstore/Cosign) with SLSA provenance attached
- ☐ Provenance verified against the expected source repository and commit SHA before publish
- ☐ SRI hashes recomputed for every third-party and first-party script and stylesheet reference
- ☐ CSP reviewed for new script-src/connect-src origins; no wildcard or unsafe-inline regressions
- ☐ Artifact published under an immutable, versioned URL (no overwrite of an existing version)
- ☐ CI/CD secrets used in this pipeline are scoped to this pipeline and rotated since last release
- ☐ Rollback target identified: previous known-good version, its hash, and its deploy command
- ☐ Release note drafted for the public status/transparency channel

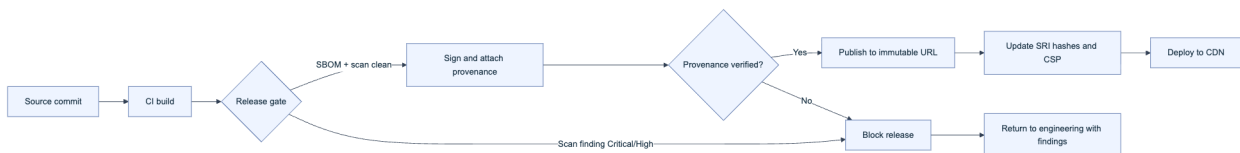


Figure 6. The release gate as a hard stop. Every branch that reaches Deploy has passed scanning, signing, and provenance verification; every failure returns to engineering rather than reaching the CDN.

16.3 Recurring Operational Controls and Cadence

Some controls are one-time gates; others decay if left unattended. A signing key that is never rotated, an access grant that outlives the employee who requested it, and an SRI hash that nobody re-audits after a silent CDN migration are all controls that were correct on day one and wrong by day two hundred. Assign each recurring control an explicit cadence and an owner, not an implicit “someone will notice.”

Control	Frequency	Trigger	Owner
Dependency vulnerability and malware scan	Continuous (every PR) plus daily scheduled	New advisory published	Application engineering
SBOM regeneration and diff	Every release	Build completion	Release engineering
CDN, registry, and DNS account access review	Quarterly	Calendar	Platform / DevOps lead
Signing key and CI token rotation	Every 90 days, or immediately on role change	Calendar or personnel change	Security engineering
SRI hash audit for drift or staleness	Monthly	Calendar	Front-end team
Domain and TLS certificate expiry check	Weekly	Calendar	Platform / DevOps
Third-party script inventory review	Quarterly	Calendar	Front-end + security
Incident response tabletop exercise	Semiannual	Calendar	Security engineering

The employee lifecycle events in this table (role change, offboarding) are the responsibility of the process described in the Employee Lifecycle Security Handbook (03); this table exists to make sure the front-end and supply-chain credentials are explicitly in scope of that process, not an afterthought discovered during an audit.

16.4 Control Maturity Self-Assessment

Not every organization needs, or can immediately afford, the most mature version of every control. This self-assessment borrows the four-bucket structure of NIST’s Secure Software Development Framework ([SP 800-218](#), Prepare the Organization, Protect the Software, Produce Well-Secured Software, Respond to Vulnerabilities) to give a team an honest read on where it stands and what the next investment should be, rather than treating maturity as all-or-nothing.

Practice area	Ad hoc	Managed	Optimized
Dependency management	Manual install, no scanning	CI-gated scan, SBOM per release	Continuous SBOM diffing, malicious-package feed blocks publish automatically

Practice area	Ad hoc	Managed	Optimized
Build integrity	Unsigned artifacts	Signed artifacts with provenance generated	Provenance verified at every deploy gate; non-conformant builds rejected automatically
Content integrity	Third-party scripts embedded raw	SRI on known third parties	SRI plus enforced CSP with strict-dynamic ; hash rotation automated in CI
Monitoring	Uptime checks only	CSP violation reports collected	Correlated telemetry across CSP, SBOM, transparency logs, and certificate transparency feeding one on-call rotation
Incident response	No documented plan	Documented playbook, untested	Playbook tested via tabletop or game day; mean time to detect (MTTD) and mean time to respond (MTTR) tracked and improving

A team should be able to place an honest mark in every row. The value of the exercise is not the score, it is the fact that “Ad hoc” in even one row is a concrete, fundable backlog item rather than a vague sense that things could be better.

17. Monitoring and Detection



Figure. NIST's continuous-improvement loop is the operating model for this chapter: runtime findings, dependency drift, build evidence, and delivery anomalies must become tracked changes to requirements and controls. Telemetry that never changes a policy or backlog item is observation, not improvement. Source: *NIST NCCoE, Continuous Monitoring and Improvement*, public domain (U.S. government work).

Every control in Chapter 16 can fail silently unless something is watching for the failure. This chapter builds the monitoring layer across the four signal sources that matter most for a Web3 front end: what the browser itself reports, what the dependency graph is doing, whether builds and releases remain provenance-clean, and whether the CDN, DNS, and certificates a user trusts are still the ones you issued.

17.1 Client-Side Signal: CSP Reporting and Real User Monitoring

The Content Security Policy from [Part II, §4.5](#) is not only a blocking control, it is a sensor. Every blocked script, style, or connection is a violation report the browser can send you, and a spike in violations for a resource that has never violated the policy before is often the first available signal of an injected script or a compromised third party, arriving well before a user complains. Configure the modern [Reporting API](#) rather than the deprecated `report-uri` directive alone:

```
Content-Security-Policy:
  script-src 'nonce-r4nd0m' 'strict-dynamic';
  object-src 'none'; base-uri 'none';
  report-to csp-endpoint

Reporting-Endpoints: csp-endpoint="https://reports.example.com/csp"
```

Run new or changed policies in `Content-Security-Policy-Report-Only` mode first so you can separate legitimate application behavior from real threats before you start blocking, then promote to enforcing once the noise is tuned out. Pair CSP reports with real user monitoring (RUM) that tracks JavaScript error rates and unexpected network destinations from the browser's perspective; RUM catches the class of compromise, DOM clobbering, malicious script injected client-side, that never touches your CDN logs at all because nothing on your infrastructure changed.

17.2 Dependency and SBOM Drift Monitoring

A dependency tree is not static once it is scanned at build time, it needs to be watched continuously because new advisories and newly discovered malicious packages appear against versions you already shipped. [OSV-Scanner V2](#), Google's open-source scanner against the [OSV.dev](#) database, now issues dedicated malicious-package identifiers (`MAL-` prefixed, mirroring the `CVE-` convention) as [OpenSSF's malicious-packages feed](#) ingests reports from multiple sources. Specialized tools such as [Socket](#) and [Phylum](#) focus specifically on behavioral indicators, an install script that reaches out to the network, a new maintainer publishing an unusually obfuscated diff, that a pure CVE-matching scanner misses entirely.

```
osv-scanner scan source --sbom cyclonedx.json --format sarif > osv-report.sarif
```

For organizations tracking dependencies at scale, [GUAC](#) (Graph for Understanding Artifact Composition), an OpenSSF-incubated project from Google, Kusari, Purdue, and Citi, aggregates SBOMs, SLSA attestations, vulnerability data, VEX (Vulnerability Exploitability eXchange) statements, and [OpenSSF Scorecard](#) results into a single queryable graph, answering "which of my deployed front ends transitively depend on this compromised package" in one query instead of a manual grep across every release's SBOM.

17.3 Build and Release Integrity Monitoring

Signing and provenance (Part III, §5.6 and §6.3) are only useful if something checks them on every install and publish, not only at initial adoption. [Sigstore's rekor-monitor](#) watches the public Rekor transparency log for entries tied to your signing identity, so a signing event you did not initiate, meaning your key or CI identity was used by someone else, surfaces as an alert rather than a surprise discovered weeks later; the project recommends running the monitor roughly hourly. At the deploy gate, verify that the artifact's provenance actually matches the expected source and commit before it is trusted:

```
slsa-verifier verify-npm-package <package>.tgz \
  --source-uri github.com/your-org/your-repo \
  --provenance-path provenance.intoto.jsonl
```

Also monitor the build pipeline's own configuration for silent changes. The [tj-actions/changed-files compromise \(CVE-2025-30066\)](#) worked precisely because a version *tag*, not a commit SHA, was repointed to malicious code; a workflow-diffing check that alerts on any change to a pinned action reference, or that flags any action referenced by mutable tag rather than SHA, would have caught it before the tag mattered.

17.4 CDN, DNS, and Domain Integrity Monitoring

The deploy-level surface needs monitoring independent of your own build pipeline, because an attacker who compromises the CDN or the domain never touches your CI at all. Monitor Certificate Transparency logs (via [crt.sh](#) or a commercial CT-monitoring feed) for any certificate issued for your domains that you did not request, catching a mis-issued certificate before it is used against a user. Track domain and registration expiry for every domain your front end references, including third-party CDN and analytics domains embedded in your pages, since an expired-and-re-registered domain is exactly the vector that turned [polyfill.io](#) from a trusted dependency into an attacker-controlled one. Run synthetic monitoring that periodically fetches your production pages, recomputes the hash of every loaded script, and diffs it against the SRI values you published, so drift is caught within minutes rather than the months it took the community to notice the Polyfill.io substitution.

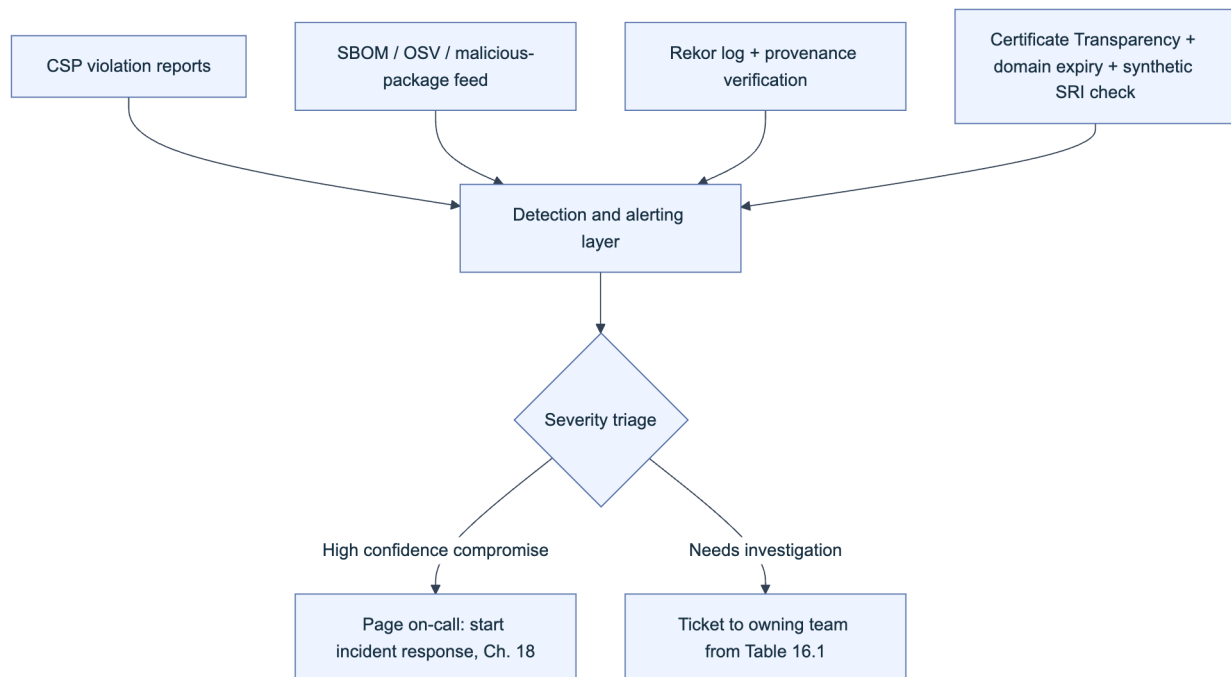


Figure 7. The four monitoring signal sources converge on one detection layer, which triages into either an immediate incident-response page or a routed investigation ticket.

18. Incident Response for Supply Chain Compromise

A plan written during an incident is a plan written under the worst possible conditions. This chapter is the front-end and supply-chain-specific complement to the organization-wide process in the Incident Response Handbook (06): read it now, adapt the templates it references, and rehearse them, so the first real incident is the second time your team has run this sequence, not the first.

18.1 Detection to Containment: The First Hours

Speed of detection is the single variable that separates a contained incident from a mass-casualty one. Contrast two real outcomes: the [singularity attack on the Nx build system](#) in August 2025 was live for just over five hours before takedown because automated monitoring and a fast community report caught it, while the [Codecov Bash Uploader compromise](#) ran undetected for roughly 65 days in 2021 because detection depended on a customer manually verifying a hash. The gap between those two numbers is the entire value proposition of Chapter 17.

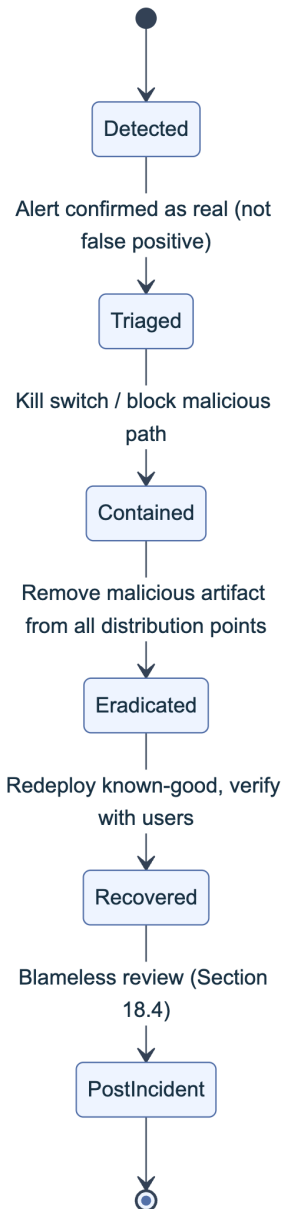


Figure 8. Incident lifecycle for a supply chain compromise, aligned to the Respond and Recover functions of *NIST SP 800-61 Revision 3*, which restructured incident handling guidance around the *NIST Cybersecurity Framework 2.0* in April 2025.

Containment for a front-end or supply-chain compromise usually means one of: pulling the malicious artifact from the CDN or registry immediately, disabling the CI/CD pipeline that could still be publishing more of it, or, if the compromise is in a widely embedded third-party script you do not control, blocking that origin at your own CSP and serving a pinned safe fallback while the upstream vendor works their own response, the exact posture Cloudflare and Fastly took during Polyfill.io by [mirroring a safe version](#) for sites that had not yet removed the dependency.

18.2 Eradication, Rollback, and Credential Rotation

Eradication is not “delete the bad file.” Rollback should always mean redeploying a previously verified, immutable artifact (Section 4.4) rather than attempting to hand-patch the compromised one, since a hand patch to code you no longer trust is itself unverified. Rotate every credential the compromised system could plausibly have reached, not just the one confirmed leaked; assume broader exposure by default, because build-time compromises routinely harvest every environment variable available, exactly what happened when Codecov’s altered uploader script exfiltrated arbitrary CI secrets rather than only Codecov’s own token, and what the [Shai-Hulud npm worm](#) did at far larger scale in September 2025, harvesting npm and GitHub tokens plus cloud metadata credentials from every CI environment it reached and using them to republish itself into more packages.

Concrete eradication steps for a front-end compromise: republish corrected SRI hashes and force a CDN edge cache purge so no user receives a cached malicious response; revoke and reissue npm, CI, and cloud provider tokens; rotate the Sigstore/Cosign signing identity if there is any chance the CI runner’s OIDC token was exposed; and if a malicious version was published under your organization’s package name, unpublish or deprecate it per the registry’s policy and file a security advisory (an npm Security Advisory or GitHub Security Advisory) so downstream consumers are warned even if they do not read your own status page.

18.3 Communication, Disclosure, and Regulatory Obligations

Draft your notification templates before you need them; an incident is the wrong time to be wordsmithing a legal disclosure from a blank page. Publicly traded U.S. companies face the SEC’s [Item 1.05 disclosure rule](#), which requires a Form 8-K within four business days of determining an incident is material, a clock that starts at the materiality determination, not at discovery. Manufacturers of software and connected products in scope of the EU’s [Cyber Resilience Act](#) face reporting obligations to ENISA that begin on September 11, 2026: a 24-hour early warning on active exploitation, a fuller 72-hour notification, and a final report within 14 days of a corrective measure or one month after the initial notification, whichever comes first.

Even teams outside either regulation’s scope should treat these windows as the de facto bar the public now expects, because the record of how fast (or slowly) an organization disclosed becomes part of the incident’s public history regardless of legal obligation, as both the Polyfill.io and Codecov timelines show. Separate two communication channels explicitly in your plan: the regulator or enterprise-customer channel, and the public user-facing channel, and make sure the public channel does not depend on infrastructure that might itself be the compromised system, you cannot reliably post an incident notice on a status page hosted on the CDN you are actively responding to a compromise of.

18.4 Post-Incident Review and Hardening

Close every incident with a blameless postmortem, published if your organization's culture and legal counsel support it. Codecov's own [April 2021 post-mortem](#) is a useful model of the format: a factual timeline, an explicit root cause, and a list of remediations, rather than a vague statement that security has been "improved." Classify the root cause against the four supply chain levels from Part I so the fix lands where the failure actually occurred; a build-level failure needs a build-level control such as pinned action SHAs, not merely a new dependency scanner that would not have caught it.

History shows that even sophisticated organizations get hit this way. Mandiant's investigation of the [3CX compromise](#) found it began as a "cascading" second-order attack, an employee's own machine was compromised through an unrelated trojanized trading application, and that foothold was used to reach 3CX's build systems months later. The [event-stream](#) and [ua-parser-js](#) npm incidents show the same lesson from the dependency side: a single compromised maintainer account or a socially engineered handoff of ownership was sufficient to reach millions of downstream installs. The differentiator across every case in this handbook is never whether the organization was targetable, everyone is, it is whether detection and response were fast enough to bound the damage. Feed every gap a postmortem surfaces directly back into the Chapter 16 checklist and the Chapter 17 monitoring coverage; an incident that does not produce a specific, dated change to those two artifacts has not actually been closed.

Key controls for Part V

- Run the release gate checklist (§16.2) as a hard stop, not an advisory step, before every front-end deploy.
- Assign an explicit cadence and owner to every recurring control (§16.3); an unowned control decays silently.
- Instrument CSP reporting, SBOM drift, transparency-log monitoring, and CT/domain monitoring as one correlated detection layer (Chapter 17), not four disconnected dashboards.
- Detection speed is the variable that matters most: compare a five-hour containment against a 65-day undetected dwell and invest accordingly.
- Rotate every credential a compromised build or CI environment could have reached, not only the one confirmed leaked.
- Pre-draft disclosure templates against the SEC four-business-day and EU CRA 24/72-hour/final windows before an incident forces you to write them live.
- Close every incident by writing a specific, dated line back into the Chapter 16 checklist or Chapter 17 monitoring coverage.

Part VI: Appendices

This part collects the reference material the rest of the handbook assumes you have on hand. Appendix A fixes the vocabulary so a term like *provenance* or *pinning* means the same thing in Chapter 4 as it does in Chapter 12. Appendix B turns the STRIDE walkthrough from Part I and the checklists scattered through Parts II through V into templates you can paste into a ticket, a design review, or a runbook. Appendix C points to the full, deduplicated source list rather than repeating every citation already given at the point of its claim.

19. Appendix A: Glossary

Terms below are grouped by the domain area where they do the most work, not strict alphabetical order, because a reader troubleshooting a CDN incident and a reader auditing a dependency tree reach for different clusters of vocabulary. Within each group, entries run alphabetically, with the acronym expanded on first appearance and the relevant chapter named so you can jump to the fuller treatment. Where an external standard or documented incident defines the term, the entry cites that source directly. Treat this appendix as a shared reference: pointing a colleague at a fixed definition here avoids the drift that happens when “provenance” means one thing to a build engineer and another to a security reviewer.

19.1 Content Delivery and Browser Security Terms

These terms describe the path from the CDN edge to the executing browser tab, the surface covered in Chapters 3, 4, and 10.

- **Cache poisoning:** storing a malicious response in a shared cache under a key that legitimate users will later request, typically by manipulating an *unkeyed input*, a request element that shapes the response without being part of the cache key. See Chapter 3.3 and [PortSwigger’s practical web cache poisoning research](#).
- **CDN (Content Delivery Network):** a distributed network of edge servers that caches and serves content close to users, discussed throughout Chapter 3.

- **Content Security Policy (CSP):** an HTTP response header that tells the browser which sources may load scripts, styles, and other resources, and can require Subresource Integrity on the scripts it allows. Defined in [CSP Level 3](#). See Chapter 4.5.
- **DOM (Document Object Model) clobbering:** an injection technique where attacker-controlled HTML elements overwrite global JavaScript variables the application assumed were script-defined, corrupting logic without a classic script payload. See Chapter 10.
- **HSTS (HTTP Strict Transport Security):** a response header, defined in [RFC 6797](#), instructing the browser to refuse any future plaintext connection to the host. See Chapter 3.4.
- **Nonce:** a per-response, unpredictable token placed in a CSP `script-src` directive and matched against a `nonce` attribute on trusted `<script>` tags, so only those scripts execute. See Chapter 4.5.
- **Prototype pollution:** a JavaScript flaw class where an attacker injects properties into `Object.prototype` or another shared prototype, altering the behavior of every object that inherits from it. See Chapter 10.
- **strict-dynamic:** a CSP `script-src` keyword that propagates trust from a nonced or hashed script to whatever it loads dynamically, replacing a brittle domain allowlist. Defined in [CSP Level 3](#). See Chapter 4.5.
- **Trusted Types:** a browser API, enforced through the `require-trusted-types-for` CSP directive, requiring DOM-injection sinks to accept only values passed through a sanitizing policy object. See the [W3C Trusted Types specification](#). See Chapter 4.5.
- **XSS (cross-site scripting):** injection of attacker script into a page viewed by another user, executing with that user's privileges and, in a wallet context, potentially reaching the signing surface. See Chapter 10.

19.2 Integrity, Signing, and Provenance Terms

These terms describe how a front end proves that delivered bytes match what was published and by whom, the core of Chapter 4.

- **Cosign:** the signing and verification client in the [Sigstore](#) project, used to attach a signature or attestation to a container image or file artifact. See Chapter 4.3.
- **Fulcio:** Sigstore's certificate authority, issuing a short-lived (roughly ten-minute) code-signing certificate that binds an ephemeral key to an OIDC-verified identity. See Chapter 4.3.
- **Immutable, versioned artifact:** a published file whose content never changes under a fixed name; any change ships under a new version or content-addressed name, so a pinned hash never goes stale silently. See Chapter 4.4.
- **Keyless signing:** a Sigstore signing mode where the signer authenticates through OpenID Connect (OIDC) instead of holding a long-lived private key; Fulcio issues an ephemeral certificate and Rekor logs the event. See Chapter 4.3.

- **Provenance:** verifiable metadata describing how, when, and by whom an artifact was built, typically including the source commit, the build platform, and the steps executed. Central to [SLSA](#). See Chapter 5.6.
- **Rekor:** Sigstore's append-only, publicly auditable transparency log, recording every signing event so an artifact owner can detect a signing they did not initiate. See Chapter 4.3.
- **Reproducible build:** a build process that, given the same source and environment, produces bit-for-bit identical output every time, letting an independent party verify a published binary against its claimed source. See Chapter 6.1.
- **Sigstore:** an open-source project providing keyless code signing, transparency logging, and verification tooling for software artifacts. See [docs.sigstore.dev](#) and Chapter 4.3.
- **SRI (Subresource Integrity):** a browser mechanism letting a page pin the cryptographic hash of an external script or stylesheet through the `integrity` attribute, so the browser refuses the resource if the fetched bytes mismatch. Defined by the [MDN SRI reference](#). See Chapter 4.1.

19.3 Dependency and Package Ecosystem Terms

These terms describe the transitive graph of code a front end pulls in before it ever reaches the CDN, covered in Chapter 5.

- **CycloneDX:** an [OWASP-maintained SBOM standard](#) whose version 1.6, released April 2024, added Cryptographic Bill of Materials (CBOM) and attestation support. See Chapter 5.2.
- **Dependency confusion:** an attack where a package manager resolves an internal package name to a same-named, attacker-published public package instead of the intended private one, usually because public-registry resolution takes precedence. First demonstrated at scale by [Alex Birsan in 2021](#). See Chapter 5.1.
- **EPSS (Exploit Prediction Scoring System):** a score estimating the probability that a given CVE (Common Vulnerabilities and Exposures identifier) will be exploited in the wild within 30 days, used to prioritize remediation beyond raw severity. See [first.org/epss](#).
- **Lockfile:** a generated manifest (`package-lock.json`, `yarn.lock`, and equivalents) that pins every direct and transitive dependency to an exact resolved version and, ideally, an integrity hash. See Chapter 5.1.
- **OSV (Open Source Vulnerabilities):** a distributed vulnerability database and schema, maintained by Google's OSV project, aggregating advisories across ecosystems for machine-readable scanning. See [osv.dev](#) and Chapter 5.3.
- **SBOM (Software Bill of Materials):** a structured, machine-readable inventory of every component, direct and transitive, in a software artifact, along with its version and supplier. See Chapter 5.2.

- **SPDX (Software Package Data Exchange)**: an SBOM standard, published as [ISO/IEC 5962:2021](#) and reaching version 3.0 in [April 2024](#) with profile-based structuring for security and AI use cases. See Chapter 5.2.
- **Transitive dependency**: a package your project did not declare directly but which one of its direct dependencies requires, often extending several levels deep and, in a typical npm project, numbering in the thousands. See Chapter 5.1.
- **Typosquatting**: publishing a package under a name that closely resembles a popular one, relying on a misspelling or misread to install the malicious version instead. See Chapter 5.1.
- **VEX (Vulnerability Exploitability eXchange)**: a companion document to an SBOM stating whether a known vulnerability in a listed component is actually exploitable in the product as shipped, using status values such as *not affected*, *affected*, *fixed*, or *under investigation*. See [CISA's VEX minimum requirements](#) and Chapter 5.2.

19.4 Build, CI/CD, and Deployment Terms

These terms describe the pipeline and the deploy-time controls that get an artifact from source to CDN, covered in Chapters 3.4 and 6.

- **CAA (Certification Authority Authorization)**: a DNS record, defined in [RFC 8659](#), restricting which certificate authorities may issue a TLS certificate for a domain. See Chapter 3.4.
- **CI/CD (Continuous Integration / Continuous Deployment)**: the automated pipeline that builds, tests, and deploys code on every change, and the build-level trust boundary covered throughout Chapter 6.
- **DNSSEC (Domain Name System Security Extensions)**: IETF extensions, originating in [RFC 4033](#), that cryptographically sign DNS records so a resolver can detect tampering or spoofed responses; treated in full in the DNS and Hosting Handbook (02).
- **Pinned action**: a CI/CD workflow reference to a third-party action or script by immutable commit SHA rather than a mutable tag such as `v1` or `latest`, closing the substitution window a compromised tag would otherwise open. See Chapter 6.4.
- **Runner**: the compute environment, self-hosted or provider-managed, that executes CI/CD jobs and holds whatever secrets those jobs are granted for the run's duration. See Chapter 6.2.
- **Secret scanning**: automated detection of credentials, API keys, or private key material committed to source control or exposed in build logs, ideally run pre-commit and again in CI. See Chapter 6.2.
- **SLSA (Supply-chain Levels for Software Artifacts)**: a framework defining graduated build and source integrity levels, from no guarantees up to a hardened, isolated build platform producing signed provenance; version 1.2, approved November 2025, added a Source Track alongside the existing Build Track. See [slsa.dev](#) and Chapter 5.6.

- **SSDF (Secure Software Development Framework):** NIST’s outcome-based set of secure development practices, published as [SP 800-218](#), developed in response to Executive Order 14028 and increasingly referenced in supplier security requirements.

19.5 Threat Modeling, Standards, and Governance Terms

These terms describe the frameworks used to structure the threat models in Chapter 2 and Part IV

- **AADAPT (Adversarial Actions in Digital Asset Payment Technologies):** a MITRE threat framework, modeled on ATT&CK and launched in 2025, cataloging adversary tactics specific to digital asset and payment systems across a twelve-tactic matrix. See [MITRE’s fact sheet](#) and the [public matrix](#).
- **Blast radius:** the scope of damage a single compromised credential, host, or component can cause, used to reason about why bounding a CDN’s or a build runner’s authority limits the impact of any one breach. See Chapter 3.1.
- **MITRE ATT&CK:** a globally accessible knowledge base of adversary tactics and techniques drawn from real-world observations, used to structure threat models for general IT and cloud environments. See [attack.mitre.org](#).
- **SCSVS (Smart Contract Security Verification Standard):** OWASP’s standard defining verifiable security requirements for smart contract systems; this handbook is its delivery-side complement. See [scs.owasp.org](#) and Chapter 1.3.
- **SCSTG (Smart Contract Security Testing Guide):** OWASP’s companion guide describing test procedures for smart contract security, extended by this handbook’s dependency and build pipeline testing. See [scs.owasp.org](#) and Chapter 1.3.
- **SCWE (Smart Contract Weakness Enumeration):** OWASP’s catalog of smart contract weakness classes, which does not cover front-end or supply-chain weaknesses, the gap this handbook fills. See [scs.owasp.org](#) and Chapter 1.3.
- **STRIDE:** a threat modeling mnemonic, spoofing, tampering, repudiation, information disclosure, denial of service, elevation of privilege, introduced by Loren Kohnfelder and Praerit Garg at Microsoft in 1999 and applied to each attack surface in Chapter 2.2 and 2.3. See [Microsoft’s threat modeling documentation](#).

19.6 Web3 and Wallet-Specific Terms

These terms describe where a front-end or supply-chain compromise ultimately lands: the wallet-signing surface.

- **Blind signing:** approving a transaction in a hardware or software wallet without the wallet being able to decode and display its actual effect, forcing the user to trust the requesting front end’s representation of what they are signing. Covered in the UX Security Handbook (09).

- **IPFS (InterPlanetary File System) pinning:** ensuring a specific piece of content stays available on the IPFS network by having at least one node commit to storing it; used in this handbook as an independently controlled fallback for a compromised CDN. See ipfs.tech and Chapter 3.5.
 - **Signing surface:** the set of front-end code paths, from form input to the wallet's `eth_sendTransaction` or `personal_sign` call, that determine what a user is asked to approve; the terminal target of most attacks in this handbook.
 - **Wallet drain:** the end state of a successful front-end or CDN compromise, in which attacker-controlled code substitutes a malicious recipient or approval into a transaction the user believes is legitimate; the outcome both the Ledger Connect Kit and Polyfill.io incidents produced. See Chapter 1.1.
 - **Web3 Attack Vectors Top 15:** an OWASP catalog ranking the most significant non-contract Web3 risks, including supply-chain compromise (WA02) and UI/UX deception (WA06). See scs.owasp.org/sctop10/Web3-Attack-Vectors-Top15 and Chapter 1.3.
-

20. Appendix B: Supply Chain Threat Model Templates

The templates below are the copy-ready form of the analysis this handbook argues for in prose. Lift a table wholesale into a design review, a release ticket, or an incident channel rather than reconstructing it from memory each time. Use the decision flow below to pick a starting point; most releases touch more than one template.

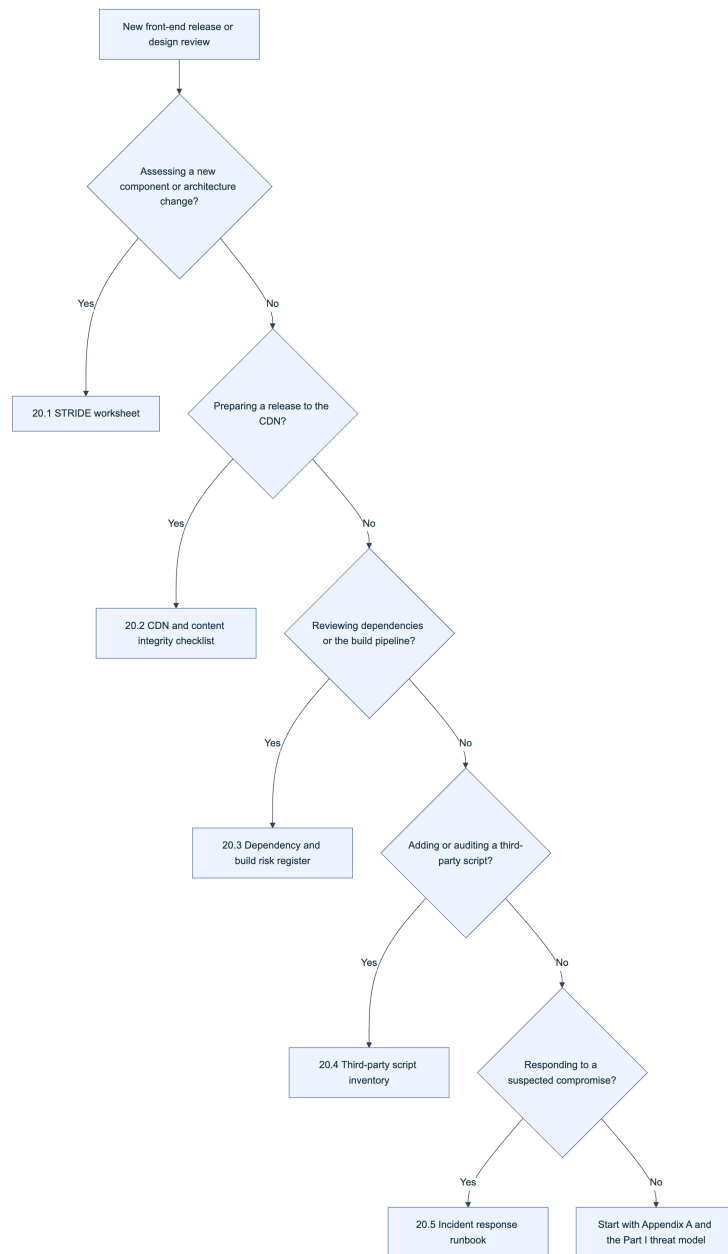


Figure 6. Template selection by trigger scenario. Most releases touch at least two branches; a release that changes a component and ships to the CDN uses both 20.1 and 20.2.

20.1 STRIDE Front-End Threat Model Worksheet

Run this worksheet whenever a new component is designed or an existing one changes trust boundaries, the same STRIDE-and-scenario method walked through in Chapter 2.3. Fill one row per component per applicable STRIDE category; not every category applies to every component, and leaving a category blank with a one-line justification is preferable to a forced entry. Revisit the worksheet at the cadence set by your release process, and re-run it in full after any incident that touched the component.

Component	STRIDE Category	Attack Scenario	Existing Control	Residual Risk (H/M/L)	Owner	Review Date
Wallet-connect script (CDN)	Spoofing	Look-alike or hijacked domain serves a modified script	SRI + CSP <code>strict-dynamic</code>	Low	Front-end lead	Quarterly
Analytics widget (third party)	Elevation of privilege	Vendor breach ships JS that reads the DOM and rewrites the signing form	CSP <code>script-src</code> scoped, no DOM access to signing component	Medium	Security review	Each vendor update
(your component)						

20.2 CDN and Content Integrity Control Checklist

Run this checklist before every production release and again on a fixed schedule (monthly is typical) independent of releases, since a third-party CDN configuration can drift without any change on your side. It operationalizes Chapters 3 and 4.

CDN and Content Integrity Checklist

- ☐ Every third-party `<script>` and `<link>` tag carries an `integrity` attribute matching the deployed version, with `crossorigin="anonymous"` set
- ☐ Every SRI-pinned resource is served from an immutable, versioned URL, never a `latest` or floating alias
- ☐ CSP is enforced with `strict-dynamic`, `object-src 'none'`, and `require-trusted-types-for 'script'`; violations report to a monitored endpoint (Chapter 17)
- ☐ TLS terminates at 1.3 (1.2 minimum) with HSTS, and CAA restricts issuance to chosen certificate authorities
- ☐ Origin rejects direct access; only the CDN's authenticated pull reaches it
- ☐ Request headers not required by the origin are stripped before the cache layer (cache poisoning defense, Chapter 3.3)
- ☐ Build artifacts are signed (Sigstore/Cosign) and verified before publish
- ☐ An independently controlled fallback (secondary CDN or IPFS pin) exists with current integrity hashes

20.3 Dependency and Build Pipeline Risk Register

Maintain this register as a living document rather than a point-in-time snapshot; the review cadence should match the update and patching policy in Chapter 5.5. Populate the SBOM Component ID column directly from your CycloneDX or SPDX output (Chapter 5.2) so the register and the machine-readable SBOM never diverge.

Inventory and integrity fields

Package / Action	Ecosystem	Direct or Transitive	SBOM Component ID	Pinned Version + Hash	License
<i>(example row)</i>	npm	Direct	pkg:npm/example@2.3.1	2.3.1 / sha512-...	MIT
<i>(your dependency)</i>					

Risk, stewardship, and disposition fields

Package / Action	Known CVEs (EPSS)	Maintainer Health	Last Reviewed	Action
<i>(example row)</i>	none open	Active, 3 maintainers	2026-Q3	None
<i>(your dependency)</i>				

20.4 Third-Party Script and Widget Inventory Template

Every third-party script or widget embedded in the front end belongs in this inventory before it ships, not discovered after the fact during an audit. This directly supports the auditing and minimization guidance in Chapter 7.3.

Script / Widget	Vendor	Purpose	Load Method	SRI Pinned?	CSP Origin Allowed	Data Accessible	Last Reviewed	Removal Candidate?
<i>(example row)</i>	Analytics vendor	Usage metrics	External <script>	Yes	script-src allowlisted	DOM read (page only), no cookies	2026-Q3	No
<i>(your script)</i>								

20.5 Supply Chain Incident Response Runbook Checklist

This checklist sequences the phases described in Chapter 18 and is meant to be rehearsed before an incident, not read for the first time during one. It is the front-end and supply-chain specific companion to the general process in the Incident Response Handbook (06); use that handbook's escalation and communication templates alongside the technical steps here.

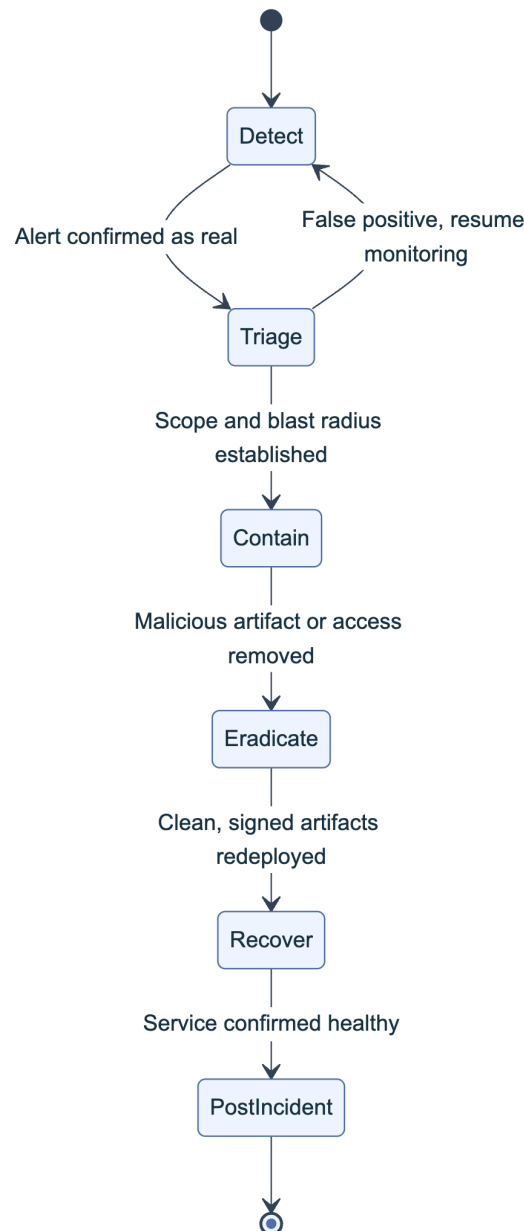


Figure 7. Supply chain incident lifecycle. Triage can return to Detect on a false positive; every other transition is forward-only and gated on the stated condition.

Supply Chain Incident Response Runbook

Detect

- ☐ Confirm the signal (CSP violation spike, SRI mismatch alert, unexpected SBOM diff, external report) against monitoring baselines (Chapter 17)

Triage

- ☐ Identify the affected supply chain level (Chapter 1.2) and the blast radius: which users, which versions, since when

Contain

- ☐ Purge or bypass the affected CDN cache; fail over to the independently controlled fallback (Chapter 3.5)
- ☐ Revoke and rotate any credential, token, or signing key with plausible exposure; if the compromise is a third-party script, remove it via a CSP update, not only by editing the embedding page

Eradicate

- ☐ Identify the entry point (compromised account, malicious dependency, poisoned cache, pinned-tag substitution) and close it
- ☐ Re-verify every artifact in the deploy path against its signed provenance before it is trusted again

Recover

- ☐ Redeploy from a known-good, signed, SBOM-matched artifact; re-issue SRI hashes and CSP nonces for anything that changed
- ☐ Confirm user-facing behavior matches the pre-incident baseline

Post-Incident

- ☐ Publish a post-mortem naming the entry point, the detection gap, and the remediation (timeline set by the Incident Response Handbook, 06), and update the risk register (20.3) and script inventory (20.4)
-

21. Appendix C: References and Further Reading

Every claim in this handbook that names a standard, a version, a date, or an incident is cited inline at the point it is made, following the convention set in Part I. Rather than duplicate that source list here, this appendix points to `references.md`, the deduplicated bibliography for the full handbook, grouped into Standards and Frameworks (OWASP SCSVS, SCSTG, SCWE, and Web3 Attack Vectors Top 15; NIST SP 800-218; SLSA; CycloneDX; SPDX; CSP Level 3; MITRE ATT&CK and AADAPT), Incidents and Case Studies (the Ledger Connect Kit compromise, the Polyfill.io takeover, the tj-actions/changed-files compromise, and the XZ Utils backdoor among them), Tools (Sigstore and Cosign, software composition analysis scanners, SBOM generators), and Further Reading (research write-ups, vendor post-mortems, and companion material that goes deeper than this handbook's scope allows).

If a link in the body of any Part goes stale, `references.md` is the place to check for an updated URL or an archive copy before assuming the underlying claim no longer holds. When you extend this handbook, add the new source to `references.md` in the same change rather than letting citations accumulate only in the body text, so the bibliography stays a complete index.

Key controls for Part 6

- Treat Appendix A as the canonical vocabulary; link to it from tickets and post-mortems rather than redefining a term inline each time.
- Copy the STRIDE worksheet (20.1) into your next architecture or design review instead of starting from a blank page.
- Run the CDN and content integrity checklist (20.2) before every production release, not only after an incident prompts one.
- Keep the dependency risk register (20.3) and the third-party script inventory (20.4) as living documents, reviewed on the cadence set in Chapter 5.5.
- Rehearse the incident response runbook (20.5) before you need it, and keep it paired with the Incident Response Handbook (06).
- Add every new citation to `references.md` in the same change that introduces it (Appendix C).

Appendix C: References and Further Reading

This appendix consolidates every external source cited across the handbook, grouped by type for ease of reference.

Standards and Frameworks

- [W3C Content Security Policy Level 3](#)
- [MDN: Subresource Integrity reference](#)
- [MDN: Content-Security-Policy report-to \(Reporting API\)](#)
- [W3C Trusted Types specification](#)
- [RFC 4033: DNS Security Introduction and Requirements](#)
- [RFC 6797: HTTP Strict Transport Security \(HSTS\)](#)
- [RFC 8659: DNS Certification Authority Authorization \(CAA\)](#)
- [CycloneDX specification overview](#)
- [SPDX overview \(ISO/IEC 5962:2021\)](#)
- [SPDX 3.0.1 specification](#)
- [Linux Foundation: SPDX 3 press release](#)
- [SLSA v1.2 specification](#)
- [SLSA v1.0: what's new \(historical; superseded by v1.2\)](#)
- [SLSA v1.2: what's new](#)
- [OpenSSF: SLSA v1.0 announcement \(historical\)](#)
- [NIST SP 800-218: Secure Software Development Framework \(SSDF\)](#)
- [NIST SP 800-218 final publication](#)
- [NIST SP 800-61 Revision 3: Incident Response Recommendations](#)
- [NTIA: Minimum Elements for a Software Bill of Materials \(SBOM\), per Executive Order 14028](#)
- [CISA: Minimum Requirements for Vulnerability Exploitability eXchange \(VEX\)](#)
- [SEC: Item 1.05 cybersecurity disclosure rule fact sheet](#)
- [Hogan Lovells: EU Cyber Resilience Act, preparing for vulnerability and incident reporting](#)
- [OpenSSF: SBOMs in the era of the CRA](#)
- [Reproducible Builds project: definition](#)
- [Debian Wiki: Reproducible Builds, About](#)
- [MITRE ATT&CK](#)
- [MITRE: AADAPT cyber threat framework for digital assets, fact sheet](#)
- [MITRE AADAPT public matrix \(GitHub\)](#)
- [FIRST: Exploit Prediction Scoring System \(EPSS\)](#)

- [OWASP Smart Contract Security \(SCS\) project home](#)
- [OWASP Web3 Attack Vectors Top 15](#)
- [OWASP Mobile Application Security \(MASVS/MASTG\)](#)
- [OWASP Cheat Sheet Series](#)
- [OWASP Cross-Site Scripting Prevention Cheat Sheet](#)

Incidents and Case Studies

- [Sansec: Polyfill.io supply chain attack disclosure, 25 June 2024](#)
- [Qualys: Polyfill.io supply chain attack, Cloudflare and Fastly mirrors](#)
- [Censys: Polyfill.io supply chain attack, web of compromised domains](#)
- [Ledger: letter from chairman and CEO Pascal Gauthier regarding the Ledger Connect Kit exploit, December 2023](#)
- [CISA Alert AA25-266A: widespread supply chain compromise impacting the npm ecosystem \(Shai-Hulud worm\)](#)
- [CISA: supply chain compromise of tj-actions/changed-files, CVE-2025-30066, March 2025](#)
- [Codecov: April 2021 Bash Uploader post-mortem](#)
- [npm blog: details about the event-stream incident](#)
- [Aikido Security: npm debug and chalk packages compromised](#)
- [BleepingComputer: popular npm library \(ua-parser-js\) hijacked to install password stealers and miners](#)
- [Nx: slngularity attack on the Nx build system, postmortem](#)
- [Trail of Bits: supply chain attacks are exploiting our assumptions \(XZ Utils backdoor\)](#)
- [Google Cloud: 3CX software supply chain compromise](#)
- [Alex Birsan: dependency confusion, how I hacked into Apple, Microsoft, and dozens of other companies \(2021\)](#)
- [The Register: British Airways breach of 2018, ICO fine](#)
- [PortSwigger Research: practical web cache poisoning \(James Kettle\)](#)

Tools and Documentation

- [OSV-Scanner \(GitHub repository\)](#)
- [OSV.dev documentation](#)
- [OSV.dev](#)
- [Google Security Blog: announcing OSV-Scanner V2](#)
- [Google blog: announcing OSV-Scanner V2 vulnerability scanning](#)
- [OpenSSF: detecting malicious packages using the OSV API](#)
- [GUAC \(Graph for Understanding Artifact Composition\)](#)
- [OpenSSF Scorecard \(GitHub repository\)](#)

- [OpenSSF: beyond scores, OpenSSF Scorecard granular structured results for custom policy enforcement](#)
- [OpenSSF: scaling up supply chain security, implementing Sigstore for container image signing](#)
- [Sigstore documentation](#)
- [Sigstore: cosign signing overview](#)
- [Sigstore blog: using rekor-monitor](#)
- [slsa-verifier \(GitHub repository\)](#)
- [diffoscope](#)
- [IPFS \(InterPlanetary File System\)](#)
- [npm docs: Trusted Publishers](#)
- [GitHub Changelog: npm trusted publishing with OIDC is generally available](#)
- [GitHub Changelog: GitHub Actions policy now supports blocking and SHA-pinning actions](#)
- [GitHub Docs: Actions security, secure use reference](#)
- [Microsoft Learn: threat modeling tool threats documentation](#)

Further Reading

- [web.dev: strict CSP guidance](#)
- [nodejs-security.com: npm ignore-scripts best practices as a security mitigation for malicious packages](#)
- [Wikimedia Commons: CDN diagram, File:NCDN - CDN.png, CC BY-SA 3.0](#)